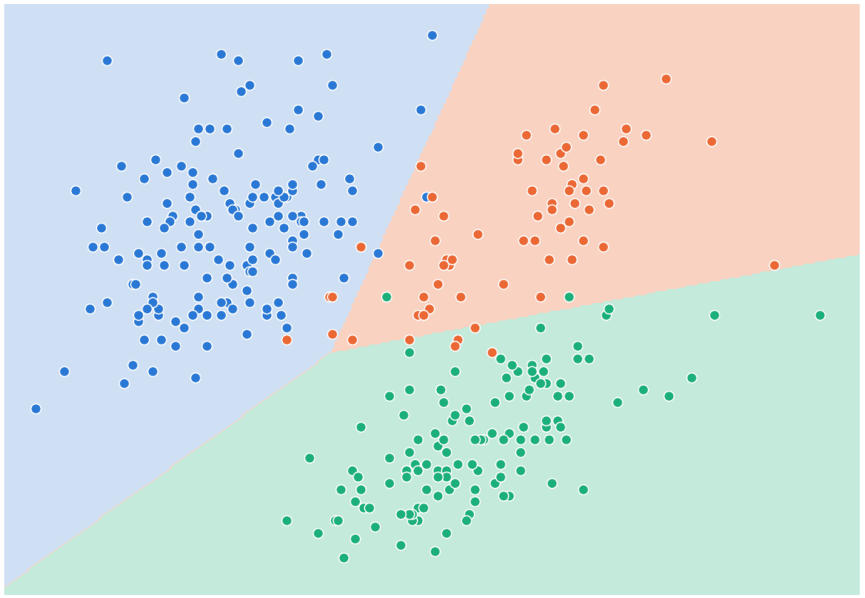
MACHINE LEARNING · WEEK 3 · LECTURE 3

Classification with Probabilities

Logistic regression · maximum likelihood · multiclass classification · Naive Bayes

Behnam Kiani
Fall Semester 2026–2027



$$P(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^\top \mathbf{x}}}$$

Three species, three probabilities, one model

Every day, a model like today's decides…



Is this message spam?

Billions of e-mails and SMS are filtered daily. Naive Bayes filters made this practical around 2002.

spam / not spam



Is this tumour malignant?

Measurements of cell nuclei from a biopsy image → probability of cancer, used to prioritise a second look.

malignant / benign



Will this user click the ad?

Large-scale logistic regression predicts click probabilities that price online advertising.

click / no click

All three answers are probabilities first, decisions second.

COURSE MAP

Where we are in the course

Week	Topics	Status
1	Learning paradigms · problem formulation · maths revision · ERM	✓ Lectures 1–2
2	Linear regression · least squares · gradient descent · ridge & lasso	✓ Lecture 2
3	**Logistic regression · maximum likelihood estimation**	**Today — Parts 01–03**
3	**Multiclass classification**	**Today — Part 04**
3	**Naive Bayes**	**Today — Part 05**
4	Bias–variance trade-off	Next week



Lab 3

- Sigmoid & log loss in NumPy
- Logistic regression by GD
- Softmax for 3 species
- Naive Bayes spam filter



Datasets

Palmer penguins · Wisconsin breast cancer · SMS Spam Collection

Last week in four ideas — and why we need them today



1 ERM

Pick a hypothesis class and a loss; minimise the average loss on the training set.

Today: new class (sigmoid), new loss (log loss)



2 Gradient descent

$w \leftarrow w - \eta \nabla J(w)$. Feature scaling; mini-batches.

Today: no closed form — GD is the only way



3 Regularisation

Ridge (L2) and lasso (L1) shrink weights and fight overfitting.

Today: needed when classes are separable



4 Least squares = MLE

Gaussian noise ⇒ maximising likelihood is minimising squared error.

Today: Bernoulli noise ⇒ log loss

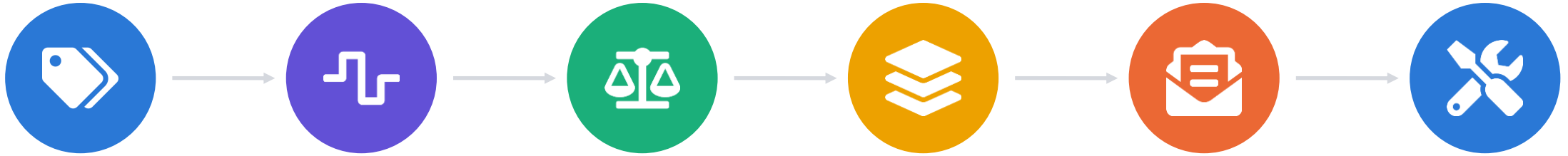
Today reuses all four ideas — only the model and the loss change.

OBJECTIVES

Today's learning objectives

1. Explain why linear regression is a poor classifier — and what the sigmoid fixes
2. Write the logistic model, read coefficients as odds ratios, draw the boundary
3. State maximum likelihood; derive the log loss and its gradient
4. Extend to K classes with one-vs-rest and softmax regression
5. Apply Bayes' theorem; build Gaussian and text Naive Bayes with smoothing
6. Choose a threshold, read a confusion matrix, compare generative vs discriminative

Roadmap for today



01 Classification

02 Logistic regression

03 Maximum likelihood

04 Multiclass

05 Naive Bayes

06 In practice

4 videos made from our own data · **5 quick questions** to answer aloud · **1 exit quiz**

WHY IT MATTERS

Classification is everywhere

Spam filtering

Naive Bayes on word counts — fast, robust, small data

Medical diagnosis

Logistic regression → probability of disease, explainable

Credit scoring

Logistic-regression scorecards; decisions must be explained

Fraud detection

Rare positives — accuracy alone is misleading

Ad-click prediction

Logistic regression on billions of sparse features

Image recognition

Softmax: the last layer of neural-network classifiers

DATA

Today's three data sets



Palmer penguins

- 219 birds: Adelie vs Chinstrap
- 342 birds: 3 species
- x = bill length, bill depth
- 2 features → we can draw it

2
features



Breast cancer (Wisconsin)

- 569 biopsies, 212 malignant
- 30 nucleus measurements
- y = malignant / benign
- Real stakes: thresholds

30
features



SMS Spam Collection

- 5,572 messages, 747 spam
- 7,387 distinct words
- x = word counts
- Text → Naive Bayes

7,387
features

PART 01

From Regression to Classification

What changes when y is a category — and why a straight line is the wrong tool



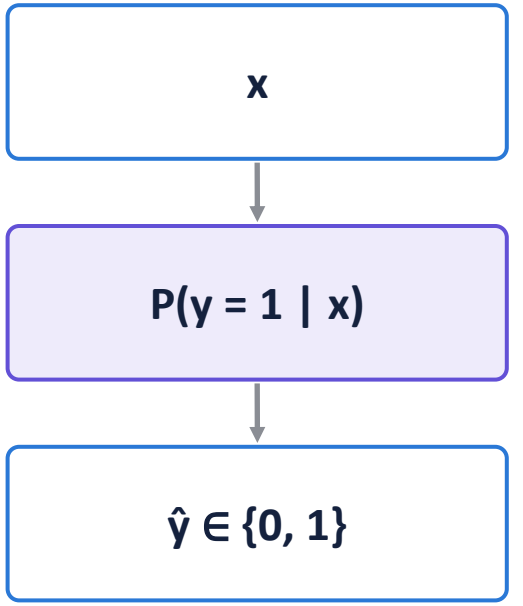
The classification problem

Set-up (same ERM recipe)

- Input $x \in \mathbb{R}^d$ · label $y \in \{0, 1\}$ (or $\{1, \ldots, K\}$)
- Classifier $h: x \mapsto$ class; 0–1 loss = $\mathbf{1}[h(x) \neq y]$
- Error rate = average 0–1 loss = 1 − accuracy

Today's key move

- Don't predict the class — predict $P(y = 1 \mid x)$
- Then decide: $\hat{y} = 1$ if $P(y = 1 \mid x) \geq t$
- Train on a smooth loss of the probability, not on the 0–1 loss

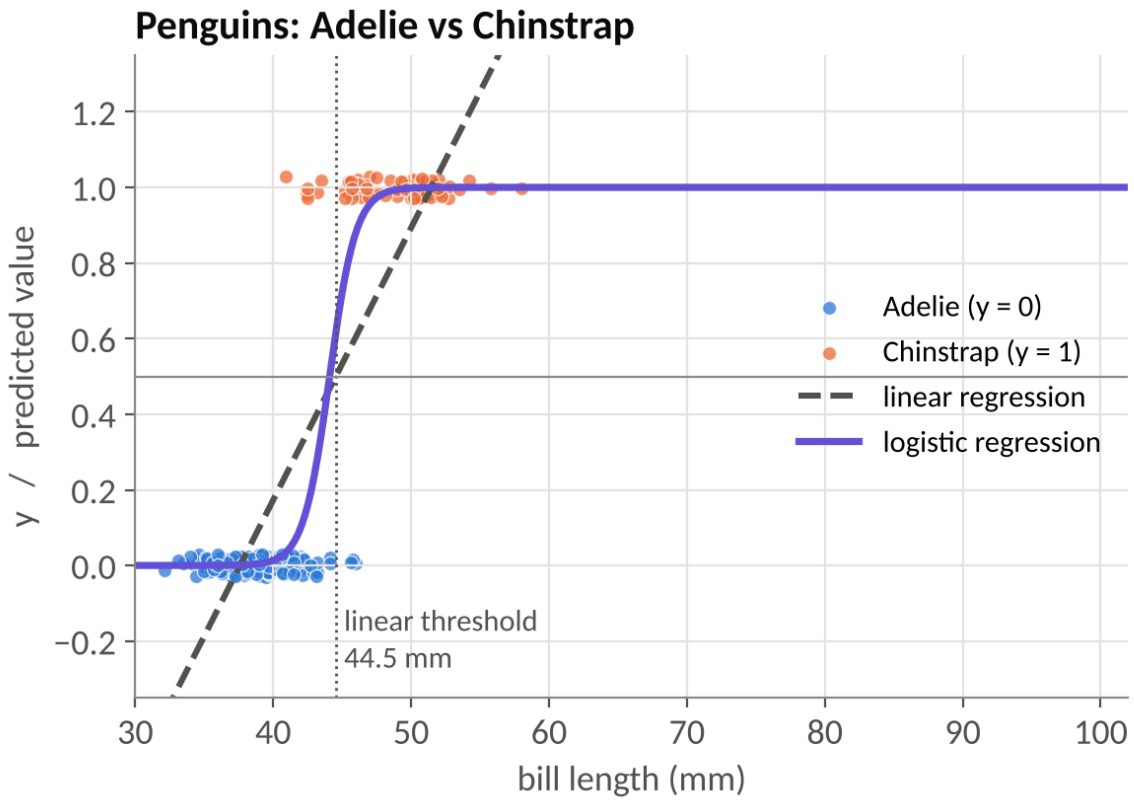


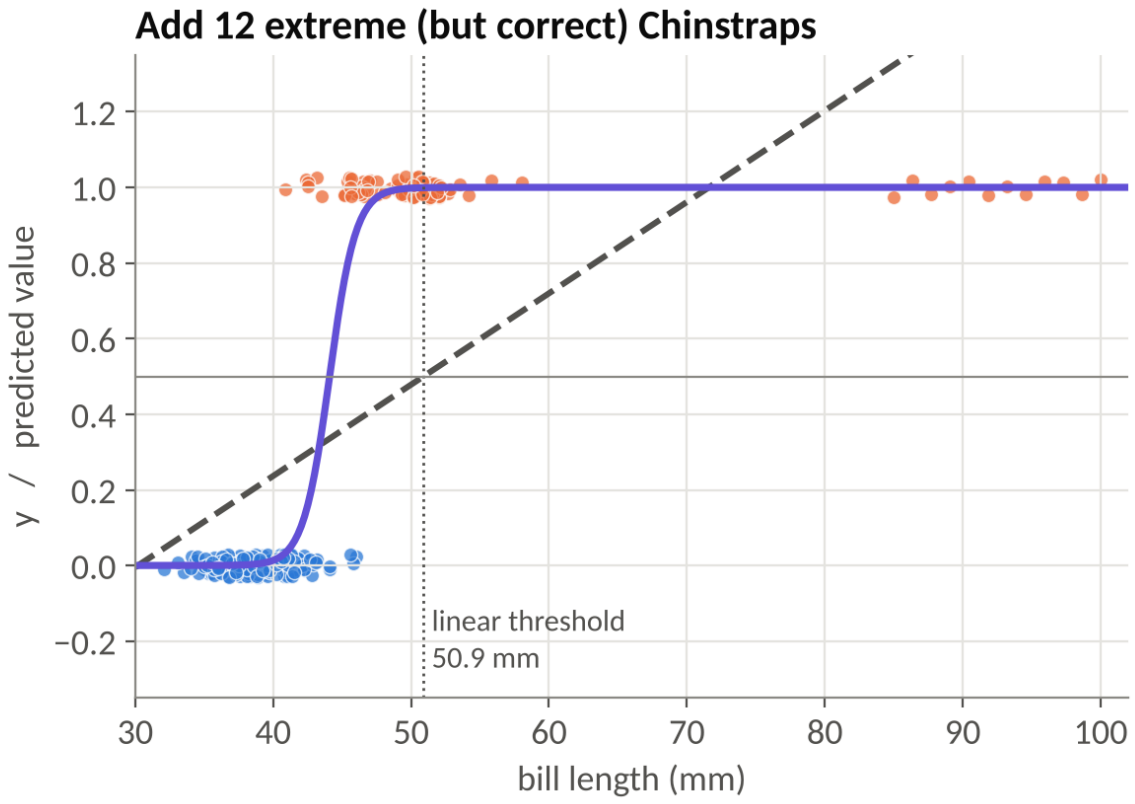
Confidence: 0.51 and 0.99 are both 'spam', but not the same

Costs: choose t from the cost of each mistake

Ranking: review the 100 most suspicious first

Why not just use linear regression?





Outputs below 0 and above 1 — not probabilities

Twelve easy points move the threshold 44.5 → 50.9 mm

The logistic curve stays in (0, 1) and barely moves

Two roads to P(y | x)

Discriminative

- Model P(y | x) directly
- Learns only what separates the classes
- Examples: logistic regression, softmax regression
- Parts 02–04

Generative

- Model P(y) and P(x | y) — how data is produced
- Flip with Bayes: P(y | x) ∝ P(x | y) P(y)
- Example: Naive Bayes
- Part 05

Both end with the same output: a probability for each class.

Quick check: classification or regression?

1	Predict the selling price of a flat in Tashkent	**Regression — y is a number**
2	Decide whether an SMS is spam	**Binary classification**
3	Predict how many times an ad will be clicked tomorrow	**Regression (a count) — but 'clicked or not?' is classification**
4	Label a biopsy: benign, malignant or uncertain	**Multiclass classification (K = 3)**

PART 02

Logistic Regression

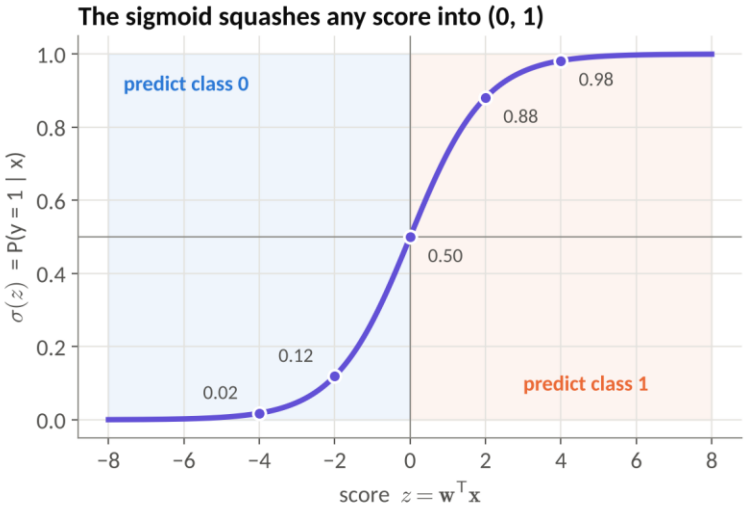
A linear score, squashed into a probability — the most widely used classifier in the world

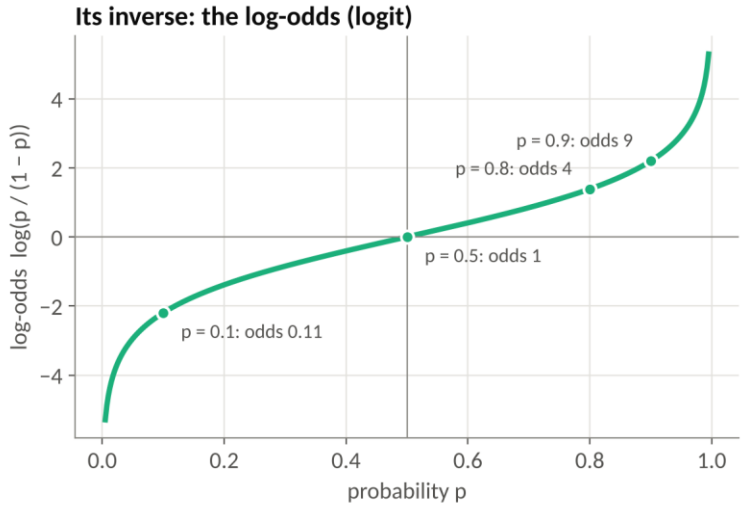


The sigmoid (logistic) function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Range:** any score $z \in \mathbb{R}$ becomes a number in (0, 1)
- **Centre:** $\sigma(0) = 0.5$; symmetric: $\sigma(-z) = 1 - \sigma(z)$
- **Saturates:** $\sigma(2) \approx 0.88$, $\sigma(4) \approx 0.98$, $\sigma(-4) \approx 0.02$
- **Inverse:** the log-odds (logit) $\log(p / (1 - p))$ maps (0, 1) back to $\mathbb{R}$



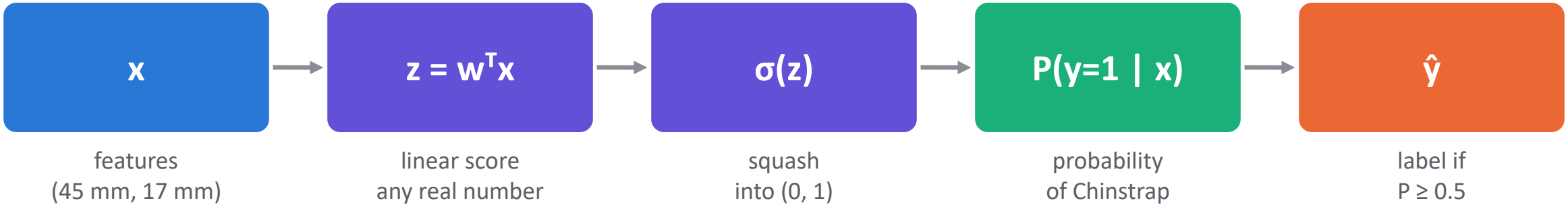


The whole trick of logistic regression

compute a linear score $z = w^T x$ (any real number), then squash it: $P(y = 1 \mid x) = \sigma(z)$.

The logistic regression model

$$P(y=1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}}$$



Same parameters as linear regression

one weight per feature + bias; scaling and regularisation carry over

Linear score, non-linear reading

the score is linear in x; only the probability is S-shaped

Odds and log-odds: what the score means

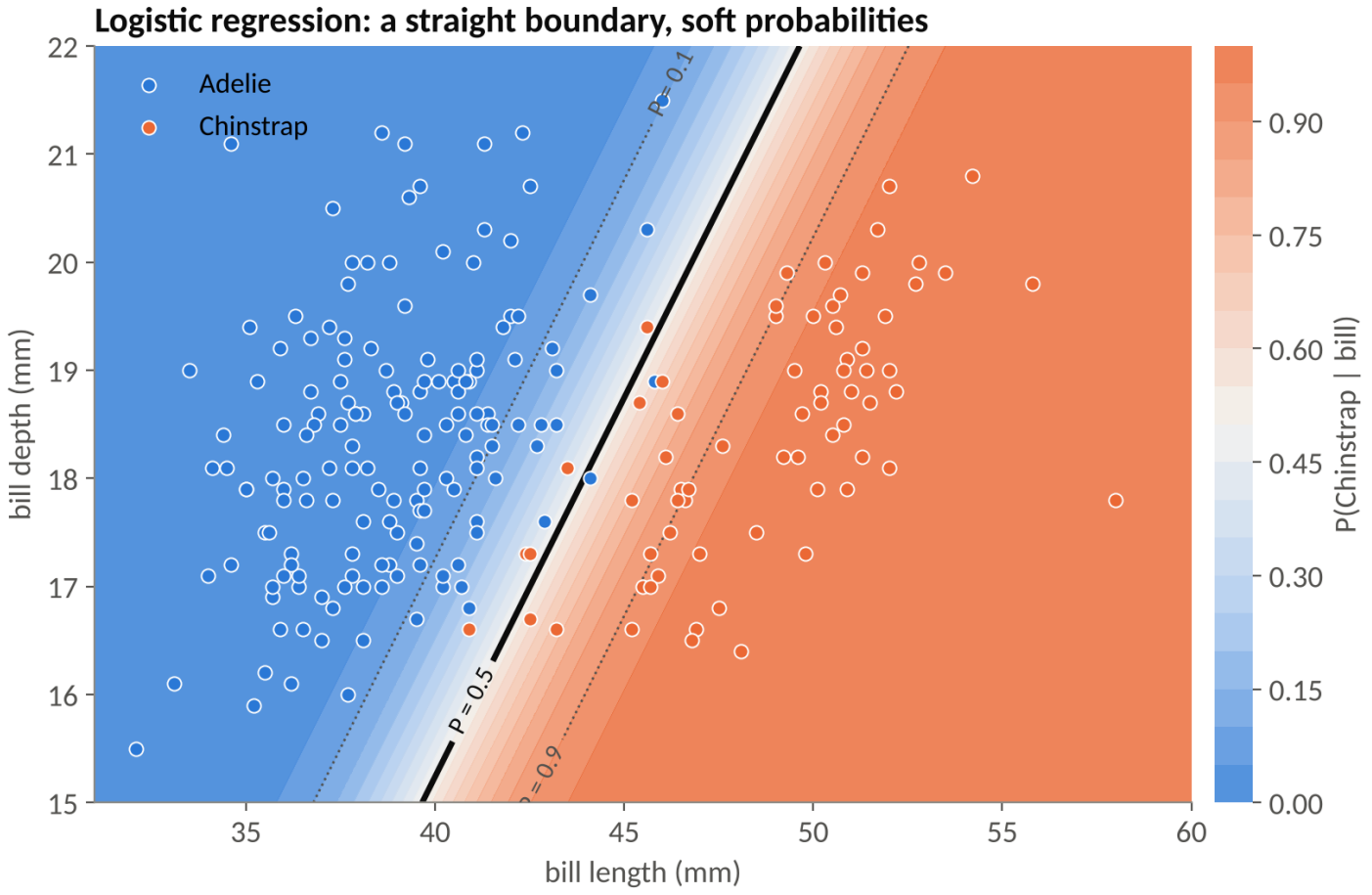
$$\log \frac{P(y=1 \mid \mathbf{x})}{1-P(y=1 \mid \mathbf{x})} = \mathbf{w}^{\top}\mathbf{x} = w_0 + w_1 x_1 + \cdots + w_d x_d$$

Probability p	Odds p / (1 − p)	Log-odds	In words
0.01	0.0101	−4.60	1 in 100
0.10	0.111	−2.20	9 to 1 against
0.50	**1**	**0**	**even**
0.75	3	1.10	3 to 1 on
0.90	9	2.20	9 to 1 on
0.99	99	4.60	99 to 1 on

Three scales

- Probability: 0 … 1
- Odds: 0 … ∞
- Log-odds: −∞ … +∞
- Only log-odds can equal a linear function of x

The decision boundary is a straight line



$$P(y = 1 \mid \mathbf{x}) \geq 0.5 \iff \mathbf{w}^{\mathsf{T}}\mathbf{x} \geq 0$$

- **Linear classifier:** $w^Tx = 0$ is a line in 2-D, a hyperplane in d-D
- **Soft edges:** P = 0.1 and 0.9 contours run parallel to the boundary
- **Weights:** length +4.17, depth −1.29 (standardised)
- **Accuracy:** 96.8% on all 219 birds — the overlap is real

One feature, real units: reading the numbers

$$\text{log-odds(Chinstrap)} = -46.26 + 1.049 \times \text{bill length (mm)}$$

44.09 mm

decision boundary:
46.26 / 1.049

40 mm bill	45 mm bill	50 mm bill
1.3%	**72.2%**	**99.8%**
log-odds -4.30	log-odds 0.95	log-odds 6.19

Ten millimetres move the prediction from ‘almost surely Adelie’ to ‘almost surely Chinstrap’.

Interpreting coefficients: odds ratios

$$\frac{\text{odds}(x_j + 1)}{\text{odds}(x_j)} = e^{w_j}$$

× 2.855 per mm

each extra millimetre of bill multiplies the odds of Chinstrap by e^1.049

Read the sign

- w > 0 → odds ratio > 1 (pushes to class 1)
- w < 0 → odds ratio < 1
- w = 0 → ratio 1: no effect

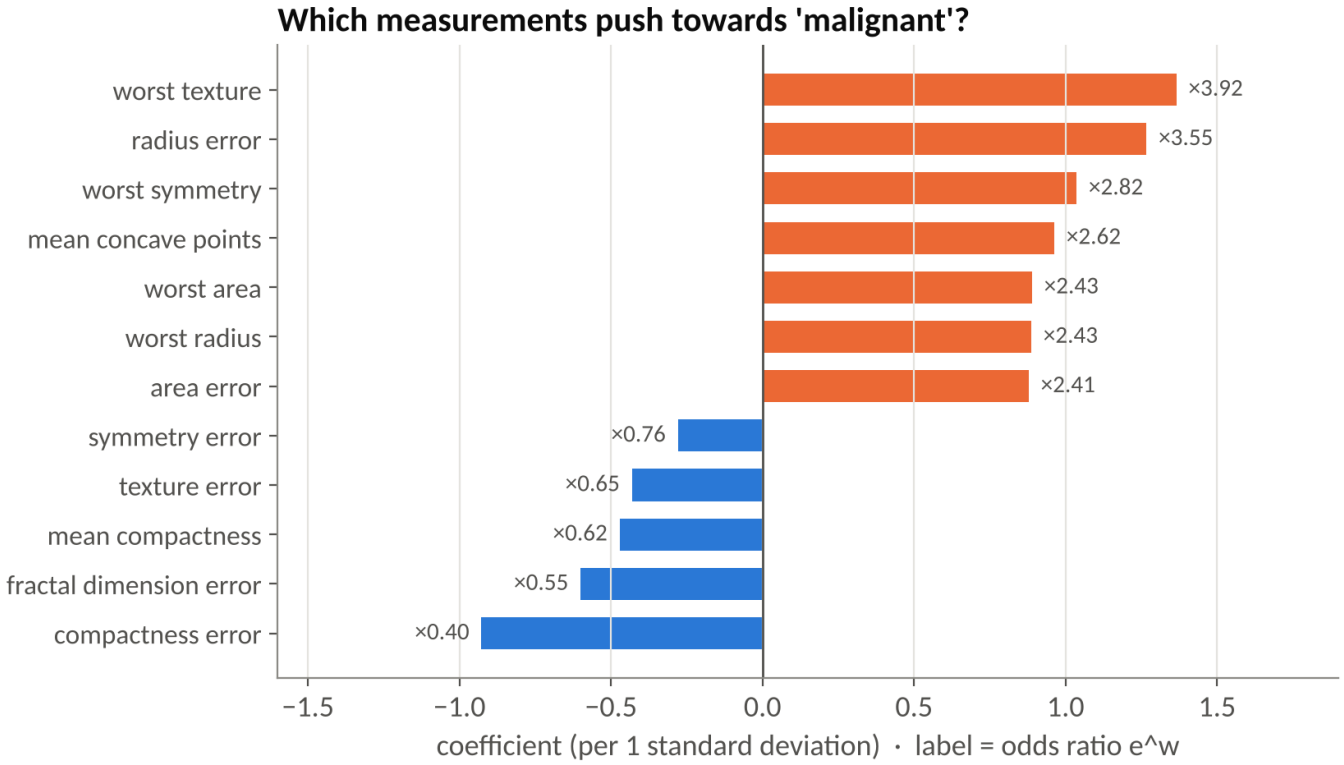
Compare fairly

- Units matter: per mm ≠ per cm
- Standardise → ratio per 1 standard deviation
- Correlated features share credit

Association, not cause

- Coefficients describe the data, not interventions
- Longer bills don't make a Chinstrap

Real use: which measurements signal cancer?



Wisconsin breast-cancer data

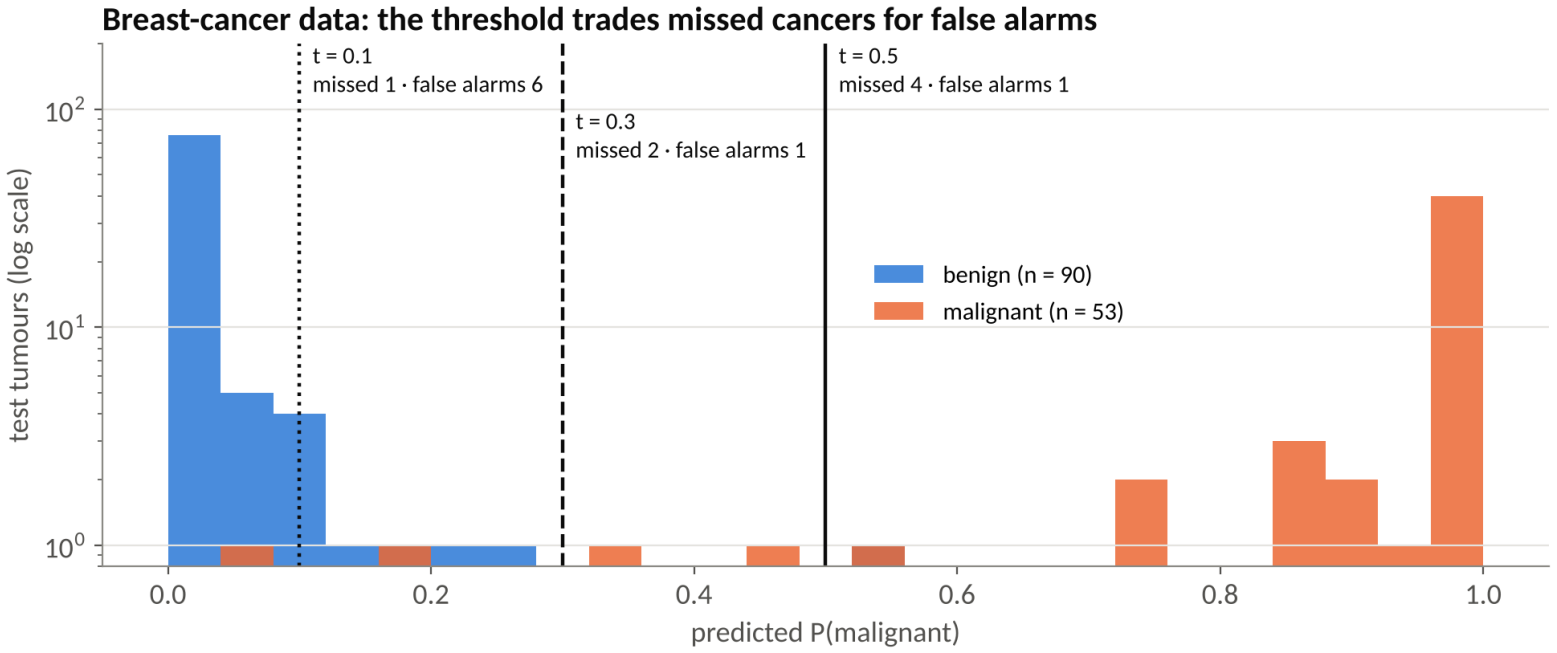
569 fine-needle biopsies; 30 features of cell nuclei measured from a digitised image (Street, Wolberg & Mangasarian, 1993).

96.5%

test accuracy (143 held-out biopsies)

Coefficients are conditional on the other 29 features — not causal, not 'protective'.

Choosing the threshold: costs, not convenience



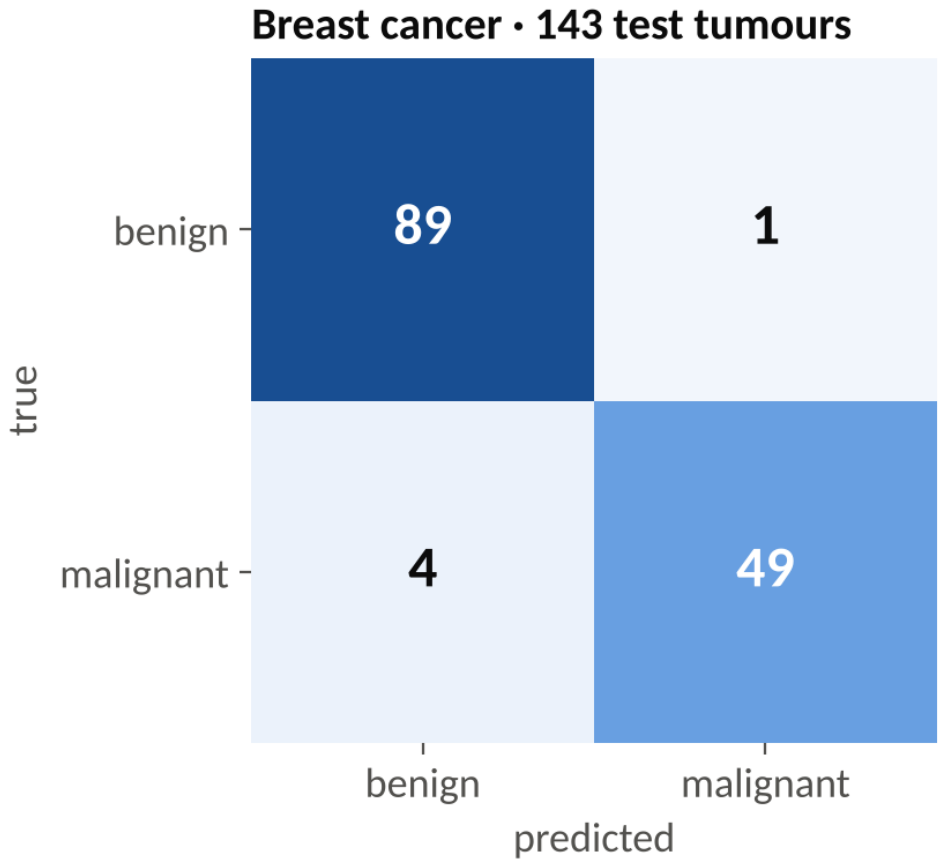
Threshold t	Missed cancers	False alarms
0.5 (default)	4	1
0.3	2	1
0.1	**1**	**6**

Decision rule

Predict positive if P ≥ C_FP / (C_FP + C_FN). Missed cancer 20× worse ⇒ t ≈ 0.05.

Same model, three decision rules. The threshold encodes the costs of each mistake.

Counting mistakes: the confusion matrix



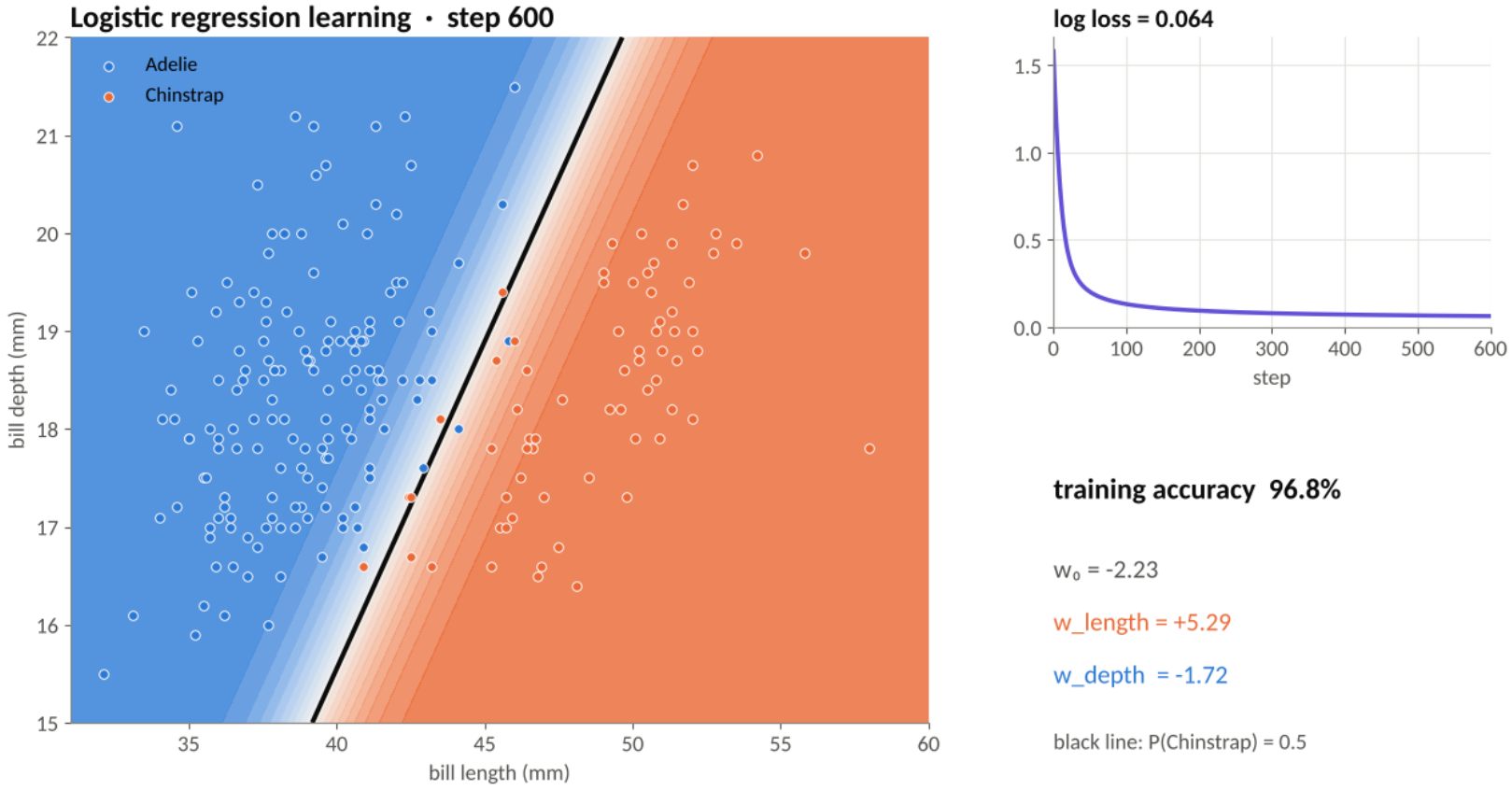
Accuracy
(TN + TP) / all
96.5%

Recall (sensitivity)
TP / (TP + FN) — cancers found
92.5%

Precision
TP / (TP + FP) — alarms that were right
98.0%

Precision–recall trade-offs and ROC curves come later in the course, with model evaluation.

Video: watch logistic regression learn



training accuracy 96.8%

w_0 = -2.23

w_length = +5.29

w_depth = -1.72

black line: P(Chinstrap) = 0.5

Watch for

- The black boundary rotates into the gap
- Colours sharpen: confidence grows
- Loss drops fast, then slowly
- Weights keep growing after the line stops

Gradient descent, η = 0.3, standardised bill length and depth, 600 steps. Click to play (≈ 14 s).

QUICK CHECK 2

Quick check: odds in your head

Model: log-odds(Chinstrap) = −46.26 + 1.049 × length. Penguin A: 43 mm. Penguin B: 45 mm. By what factor are B's odds of being a Chinstrap larger than A's?

× 8.15

e^(2 × 1.049) = e^2.098

Penguin	Log-odds	P(Chinstrap)	Odds
A · 43 mm	−1.15	0.24	0.32
B · 45 mm	+0.95	0.72	2.6

Hint

Per mm the odds are × e^1.049 ≈ 2.86, so two mm give 2.86 × 2.86.

Odds multiply by 8 — probabilities only triple. Odds ratios act on odds, not on probabilities.

PART 03

Maximum Likelihood

Where loss functions come from — and how to derive the gradient of logistic regression



Where does a loss function come from?



The maximum likelihood principle

Choose the parameters under which the data we actually observed would have been most probable.



Detective analogy

Each suspect (value of w) explains the evidence (labels) with some probability. Pick the one under whom the evidence is least surprising.

1

Model one example

$P(y_i \mid x_i; w)$

2

Multiply over examples

independence (i.i.d.)

3

Take the log

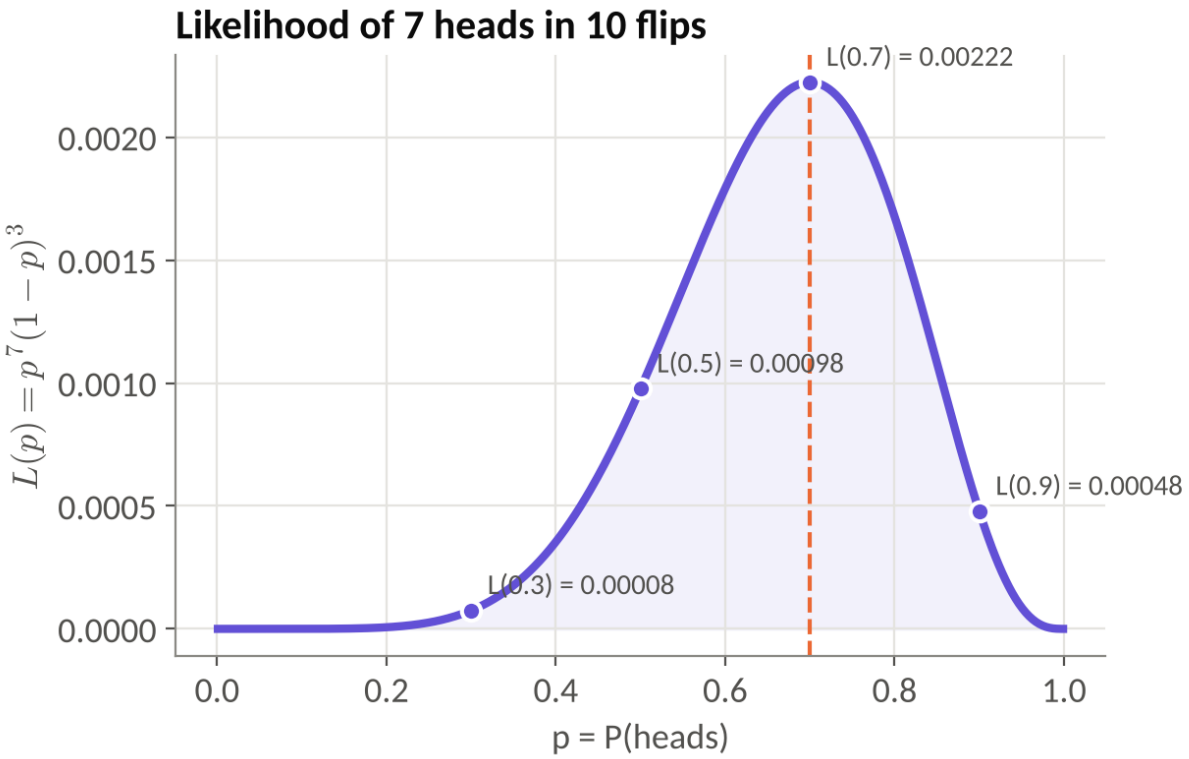
products → sums

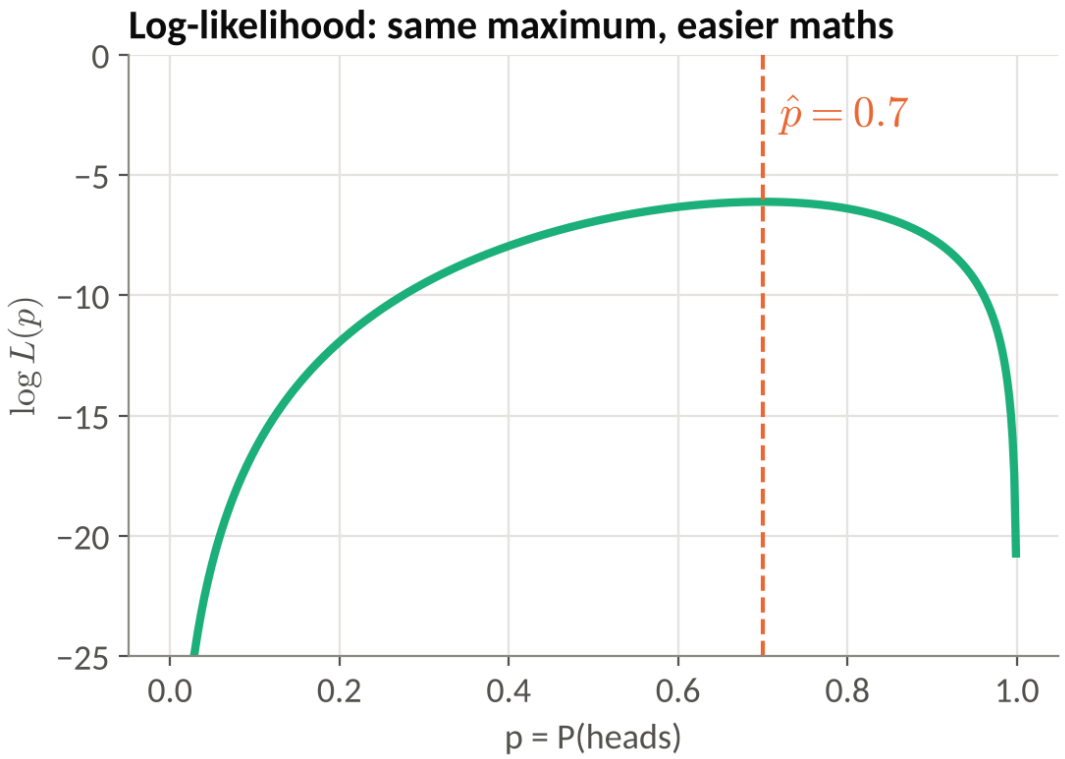
4

Maximise

= minimise −log L: that is the loss

Likelihood in one example: a biased coin

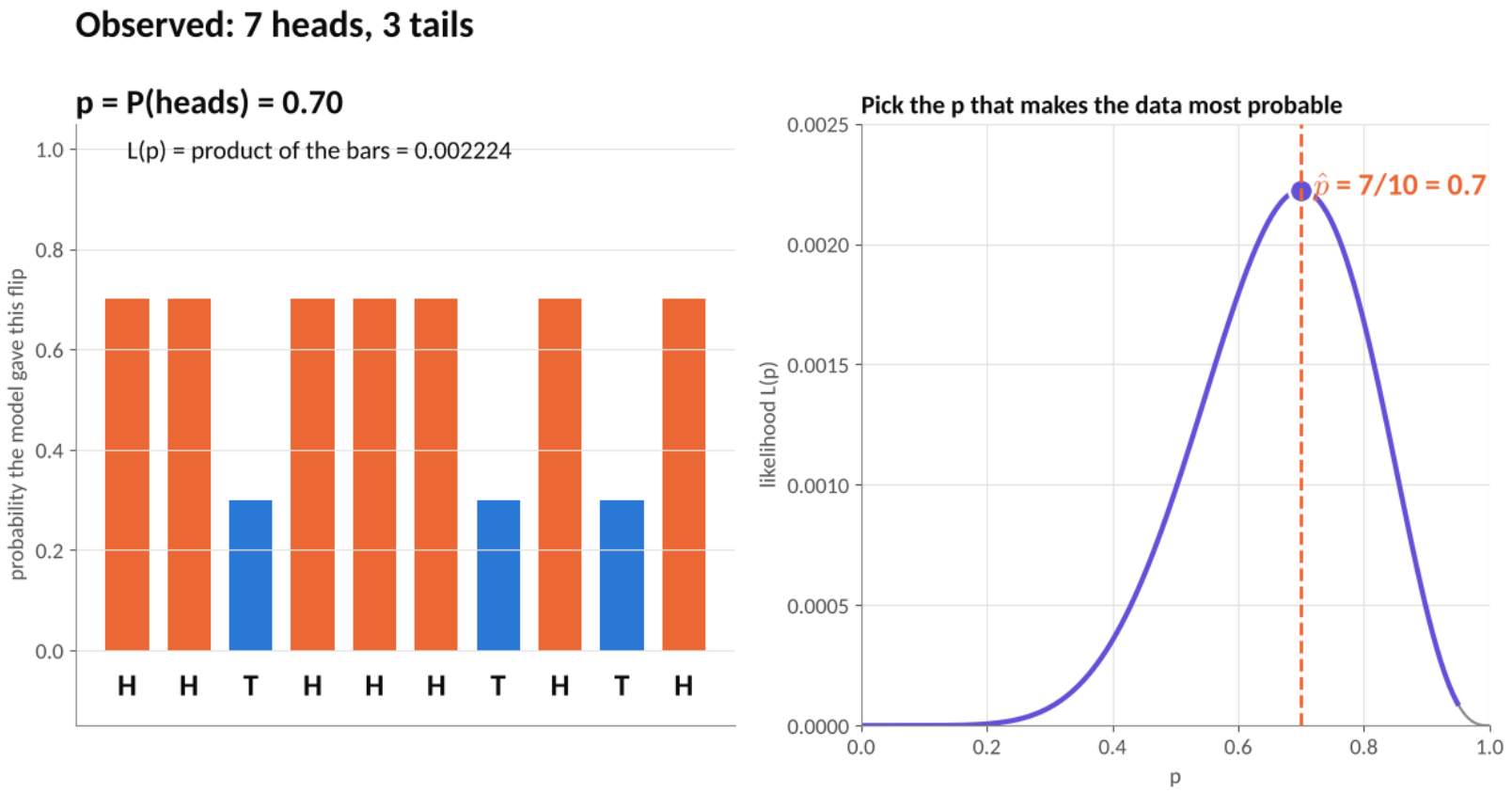




$$L(p) = p^7(1-p)^3, \quad \frac{d}{dp}\log L = \frac{7}{p} - \frac{3}{1-p} = 0 \Rightarrow \hat{p} = 0.7$$

Likelihood: probability of the data, as a function of the parameter.

Video: maximum likelihood, step by step



Watch for

- Each bar = probability of what actually happened
- Small p: heads punished
- Large p: tails punished
- Best compromise: p̂ = 0.7

The likelihood is the product of the ten bars. Click to play (≈ 12 s).

Why we always take the log

Same maximum

log is strictly increasing: arg max log L = arg max L

Products → sums

a sum over examples = an average loss, exactly what ERM needs

No underflow

computers cannot store 10^{-3000}; they can store −6931

```
0.5 ** 1000            # 9.33e-302
0.5 ** 1100            # 0.0   <- underflow
1000 * np.log(0.5)     # -693.1   <- fine
```

≈ 1,075

examples of probability 0.5 are enough to underflow a 64-bit float to exactly 0

In practice: minimise the negative log-likelihood (NLL).

Promise kept: least squares is maximum likelihood

$$y_i \sim \mathcal{N}(\mathbf{w}^\top \mathbf{x}_i,\ \sigma^2) \ \Rightarrow\ -\log L(\mathbf{w}) = \frac{1}{2\sigma^2}\sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i)^2 + \text{const}$$

Gaussian noise	→	**–log L = squared error**	*least squares (Week 2)*
Laplace noise	→	**–log L = absolute error**	*robust regression*
Bernoulli (coin) label	→	**–log L = log loss**	*logistic regression (today)*

Choosing a loss = choosing a noise model.

Logistic regression as a coin model

$$P(y \mid \mathbf{x}; \mathbf{w}) = p^{\,y}\,(1-p)^{1-y}, \qquad p = \sigma(\mathbf{w}^{\mathsf{T}}\mathbf{x})$$

$$L(\mathbf{w}) = \prod_{i=1}^{n} P(y_i \mid \mathbf{x}_i; \mathbf{w}) = \prod_{i=1}^{n} p_i^{\,y_i}(1-p_i)^{1-y_i}$$

y = 1

$p^1(1-p)^0 = p$: the probability the model gave to '1'

y = 0

$p^0(1-p)^1 = 1-p$: the probability the model gave to '0'

One coin per penguin

Each x_i has its own bias $p_i = \sigma(w^{\mathsf{T}}x_i)$; w controls all the coins at once.

The negative log-likelihood is the log loss

$$J(\mathbf{w}) = -\frac{1}{n}\sum_{i=1}^{n}[\, y_i \log p_i + (1 - y_i)\log(1 - p_i)\,]$$

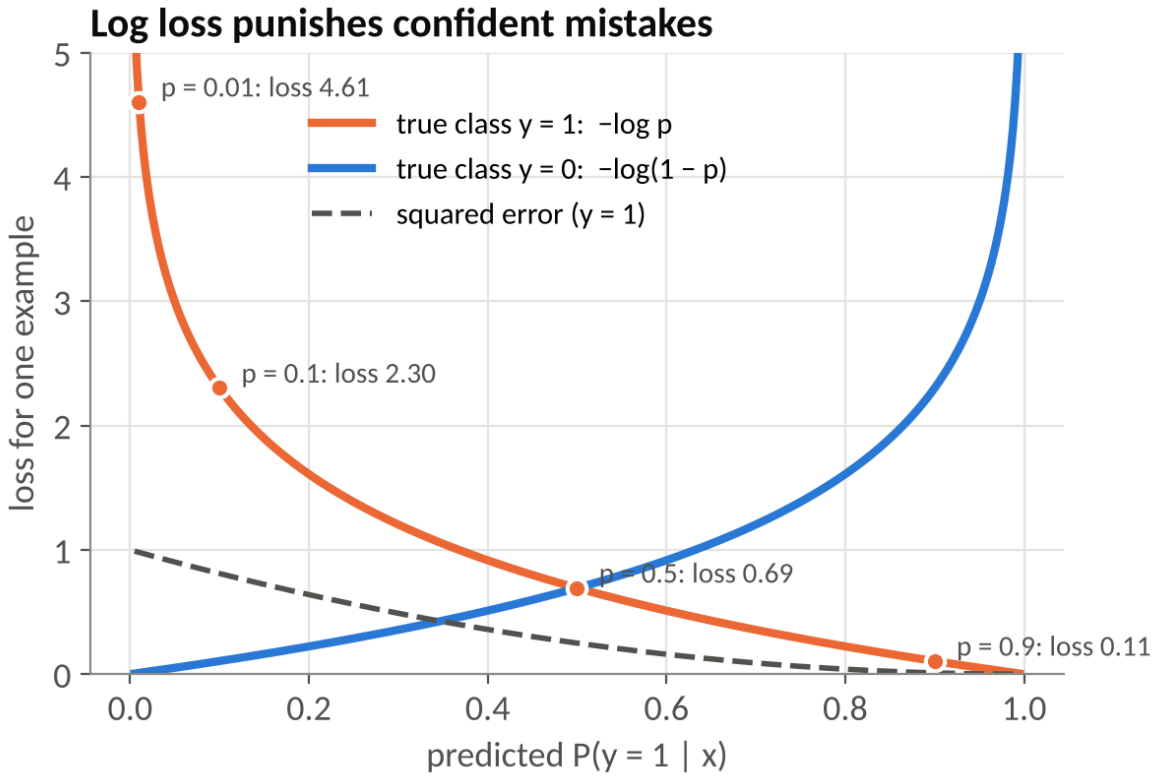
P(true class)	Loss –log p	Verdict
0.99	0.01	confident and right
0.90	0.11	right
0.50	0.69 (= log 2)	doesn't know
0.10	2.30	wrong
0.01	**4.61**	**confidently wrong**

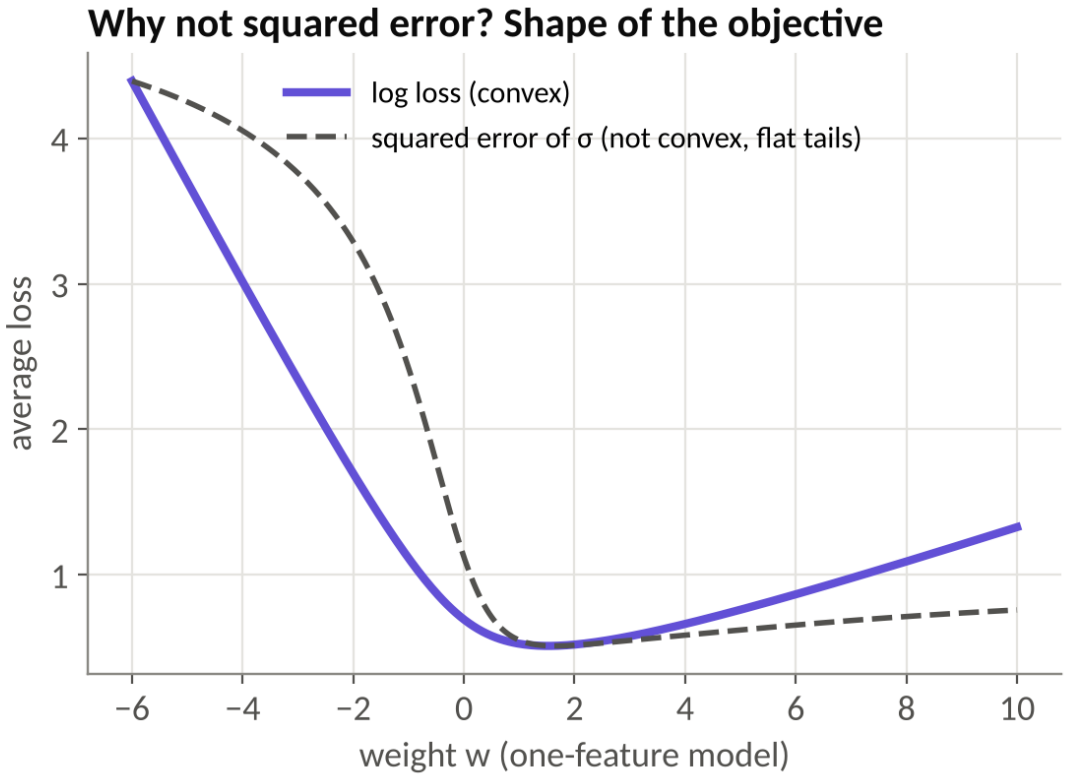


Three names, one loss

- Log loss
- Binary cross-entropy (PyTorch: BCELoss)
- Negative log-likelihood (NLL)
- Gentle with doubt, brutal with confident mistakes

Why not squared error on probabilities?





Confidently wrong: squared ≤ 1, log loss → ∞

Sigmoid + squared: flat tails stall learning

Log loss is convex: GD finds the minimum

Deriving the gradient with the chain rule

$$\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$$

Chain rule for one example:

$$\frac{\partial \ell_i}{\partial \mathbf{w}} = \frac{\partial \ell_i}{\partial p_i} \cdot \frac{\partial p_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial \mathbf{w}} = \left(-\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i} \right) p_i(1 - p_i)\,\mathbf{x}_i = (p_i - y_i)\,\mathbf{x}_i$$

$$\nabla J(\mathbf{w}) = \frac{1}{n} X^\top (\sigma(X\mathbf{w}) - \mathbf{y})$$

Gradient = (1/n) Σ (prediction − truth) × input

The magic cancellation

p(1 − p) from the sigmoid cancels the denominators of the log loss: what's left is prediction error × input.

Look familiar? The same gradient as linear regression

Linear regression (Week 2)

$$\nabla J_{\text{linear}}(\mathbf{w}) = \frac{2}{n} X^\top (X\mathbf{w} - \mathbf{y})$$

Logistic regression (today)

$$\nabla J(\mathbf{w}) = \frac{1}{n} X^\top (\sigma(X\mathbf{w}) - \mathbf{y})$$

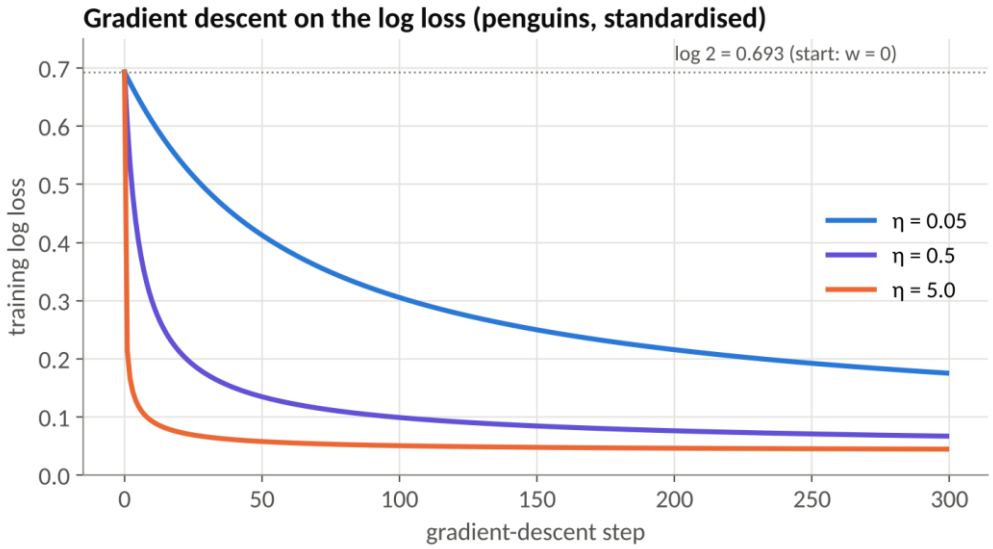
X^T × (prediction – truth) — the gradient of every NLL with the right output distribution

Gaussian → linear regression · Bernoulli → logistic regression · Categorical → softmax (Part 04) · and every neural-network output layer.

Gradient descent for logistic regression in NumPy

```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

Xs = (X - X.mean(0)) / X.std(0)    # standardise
A = np.c_[np.ones(n), Xs]          # bias column
w = np.zeros(A.shape[1]); eta = 0.5
for step in range(300):
    p = sigmoid(A @ w)             # probabilities
    grad = A.T @ (p - y) / n       # (1/n) Xᵀ(σ(Xw) -
y)
    w = w - eta * grad
    loss = -np.mean(y*np.log(p) + (1-y)*np.log(1-p))
```



- **Start:** log 2 = 0.693, because w = 0 ⇒ p = 0.5
- **η:** tolerates large steps (σ′ ≤ ¼) — still tune it
- **Stability:** exp(−z) can overflow — clip z or use a stable sigmoid

Perfectly separable data: when weights run to infinity

scikit-learn C	Penalty λ = 1/C	Largest |w|	Training accuracy
1 (default)	1	2.9	99.6%
100	0.01	8.7	100%
10^4	10^{-4}	24.4	100%
10^6	10^{-6}	43.3	100%
10^8	**10^{-8}**	**63.4**	**100%**

Adelie vs Gentoo (274 birds), standardised bill length and depth — perfectly separable.

$$J_\lambda(\mathbf{w}) = J(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2 \qquad (\text{scikit-learn}: \; C = 1/\lambda)$$

What goes wrong

- Loss → 0 but never reaches it
- Weights grow forever
- Overconfident probabilities

The cure: regularise

- Same L2 penalty as ridge
- scikit-learn default C = 1
- Larger C = weaker penalty

Regularisation is a prior: MLE vs MAP

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \; P(\text{data} \mid \theta)\, P(\theta)$$

−log posterior = loss + penalty

Gaussian prior on w

→ L2 penalty

ridge, regularised logistic regression

Laplace prior on w

→ L1 penalty

lasso — sparse weights

Prior on counts

→ add α to every count

Laplace smoothing in Naive Bayes (Part 05)

3 heads in 3 flips: MLE $\hat{p} = 1$ (tails impossible?!) · with one imaginary head and tail: (3+1)/(3+2) = 0.8

Quick check: compute the log loss

1. A spam message; model says P(spam) = 0.9

   $-\log 0.9 = 0.105$

2. A ham message; model says P(spam) = 0.4

   truth got 0.6 → $-\log 0.6 = 0.51$

3. A ham message; model says P(spam) = 0.99

   truth got 0.01 → $-\log 0.01 = 4.61$

Use the probability given to the TRUE class. The confident mistake costs 44× the good prediction.

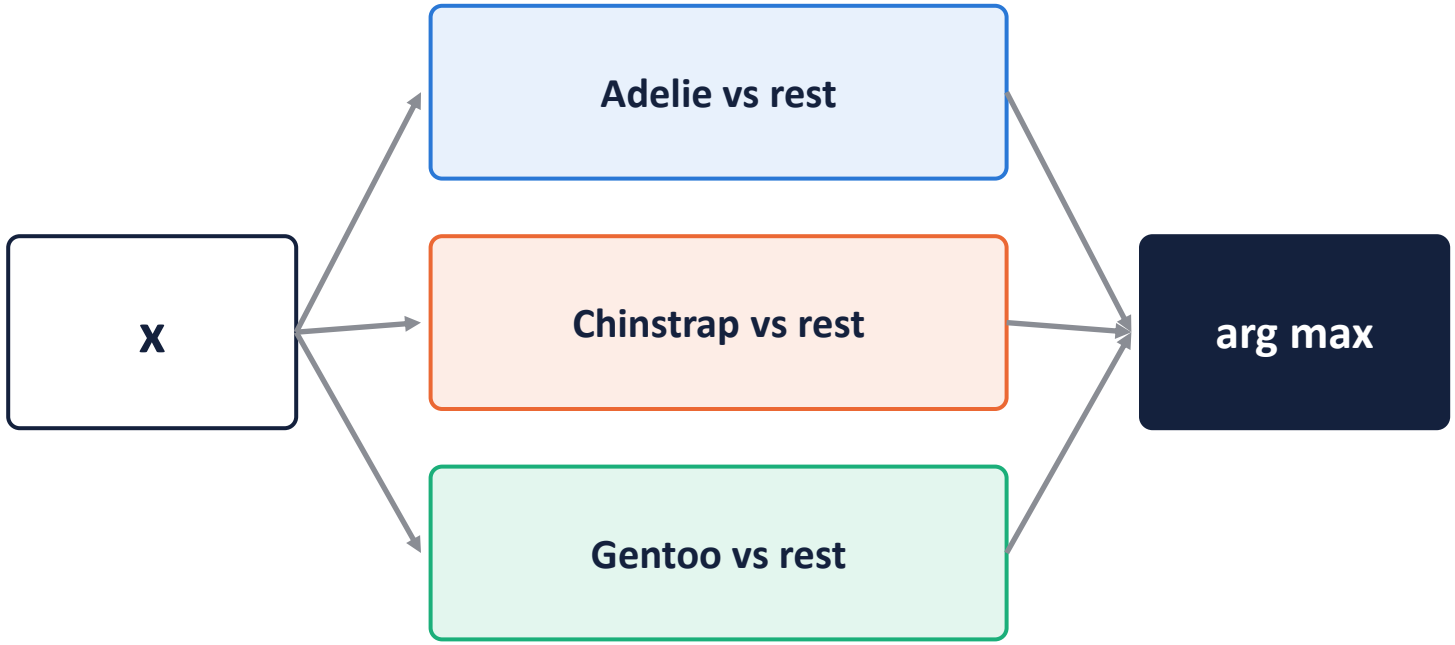
PART 04

Multiclass Classification

From yes/no to K classes: one-vs-rest, softmax and cross-entropy



Strategy 1: combine binary classifiers



$$\hat{y} = \arg\max_k \ \sigma(\mathbf{w}_k^\top \mathbf{x})$$

Classes K	One-vs-rest	One-vs-one
3 (penguins)	3	3
10 (digits)	10	45
1,000 (images)	1,000	499,500

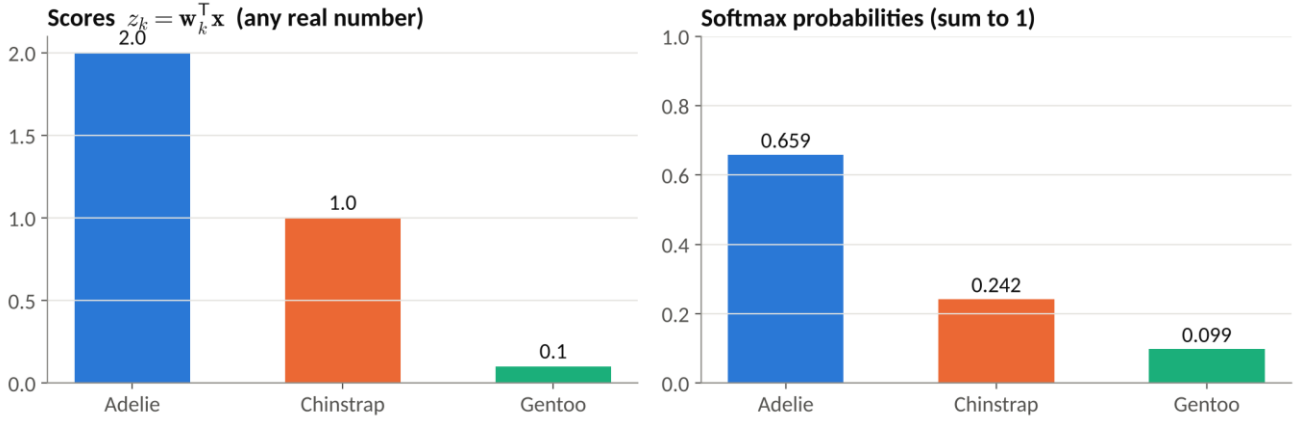
Weak points

- Scores don't sum to 1
- 'Rest' class is much bigger
- Ambiguous regions where boundaries cross

Strategy 2: the softmax function

$$P(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}_k^\top \mathbf{x}}}{\sum_{j=1}^{K} e^{\mathbf{w}_j^\top \mathbf{x}}}$$

- **Exponentiate:** every score becomes positive
- **Normalise:** divide by the sum → probabilities add to 1
- **Soft max:** a smooth, differentiable version of 'pick the largest'



$e^2 : e^1 : e^{0.1} = 7.39 : 2.72 : 1.11 \rightarrow \div 11.2$

Shift-invariant

adding a constant to every score changes nothing

K = 2

softmax = sigmoid of the score difference

Everywhere

last layer of neural nets; LLM next-word probabilities

Softmax regression: model, loss and gradient

$$J(W) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K}\mathbf{1}[y_i = k]\log p_{ik}$$

$$\nabla_W J = \frac{1}{n} X^\top (P - Y)$$

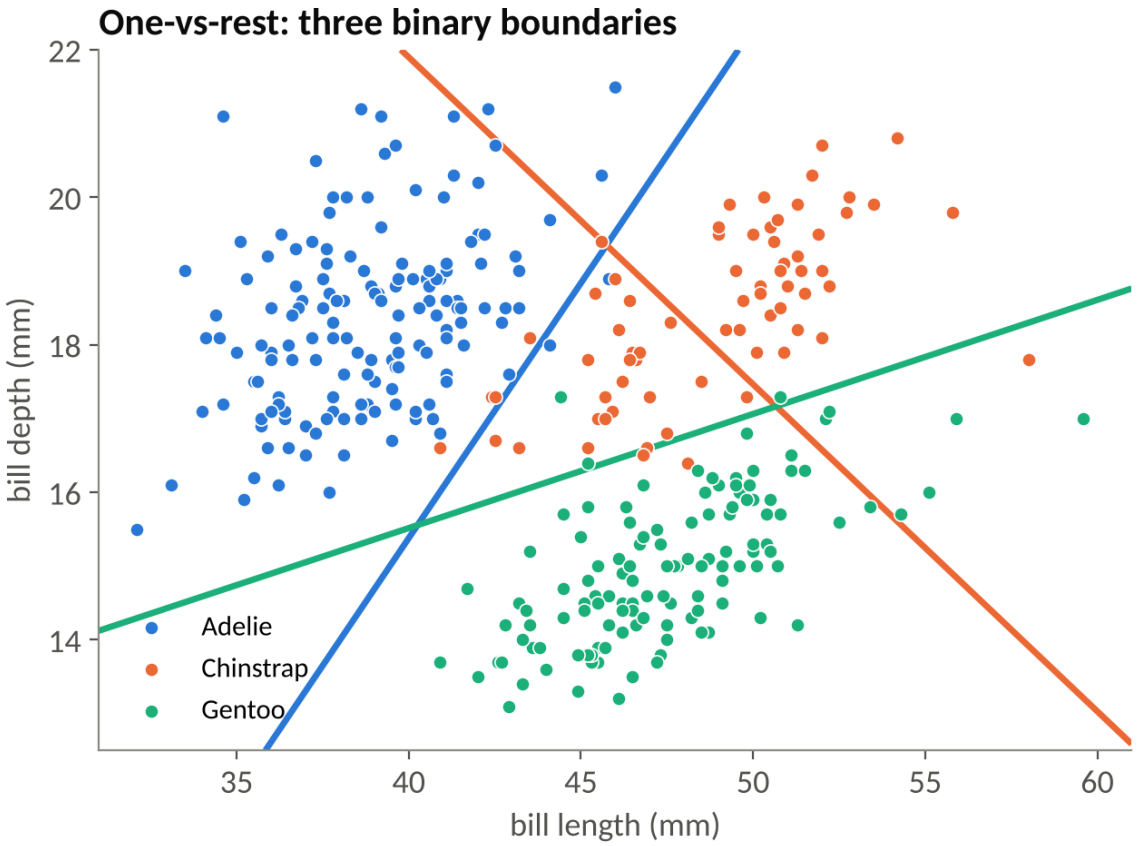
- **Parameters:** W = [w_1 … w_K], (d + 1) × K numbers
- **Also called:** multinomial logistic regression; ‘categorical cross-entropy’ loss

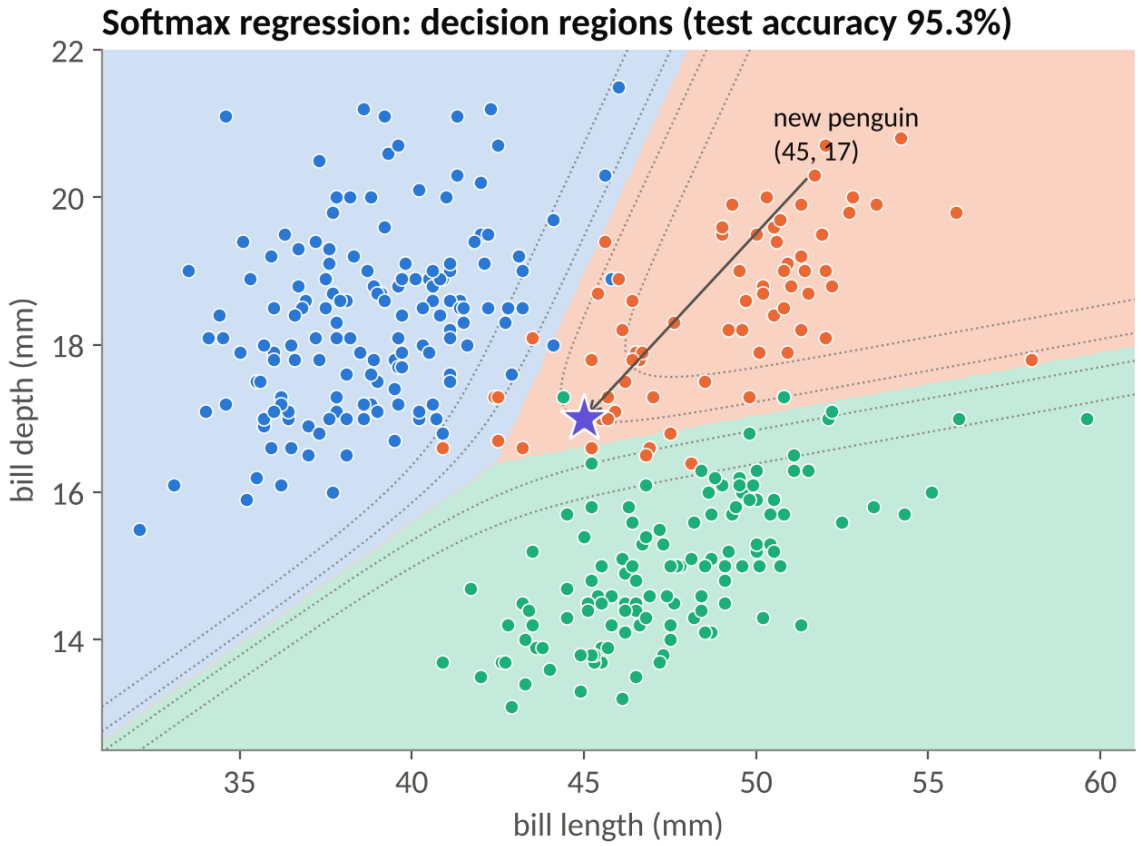
Species	One-hot y
Adelie	[1, 0, 0]
Chinstrap	[0, 1, 0]
Gentoo	[0, 0, 1]

Same recipe, third time

- Label = K-sided die (categorical)
- Loss = −log P(true class)
- Gradient = X^T(prediction − truth)
- Code: sigmoid → softmax, y → one-hot Y

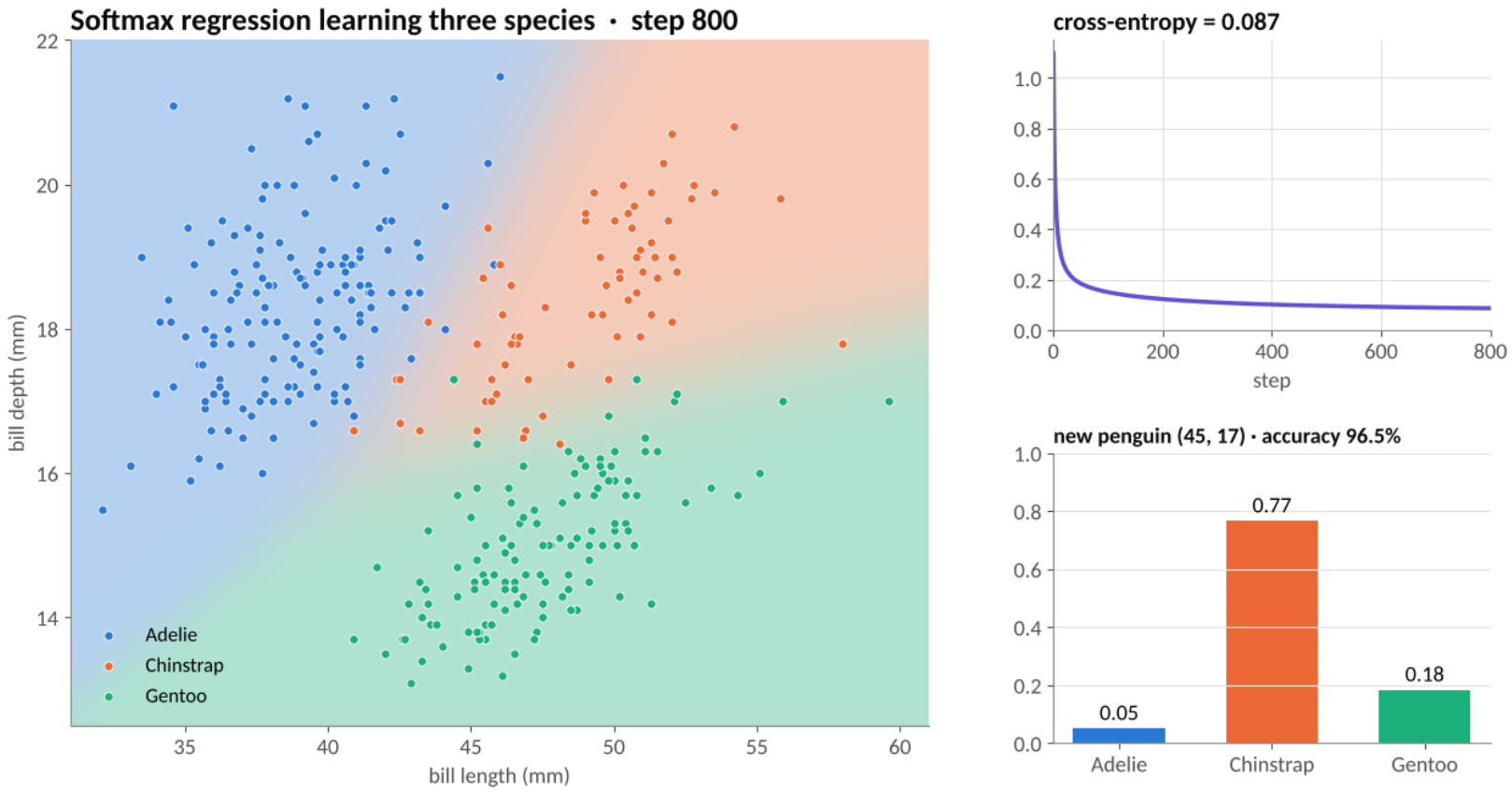
Three species: one-vs-rest vs softmax





★ **new penguin (45, 17):** Adelie **0.16** Chinstrap **0.55** Gentoo **0.29**

Video: softmax regression learning three species

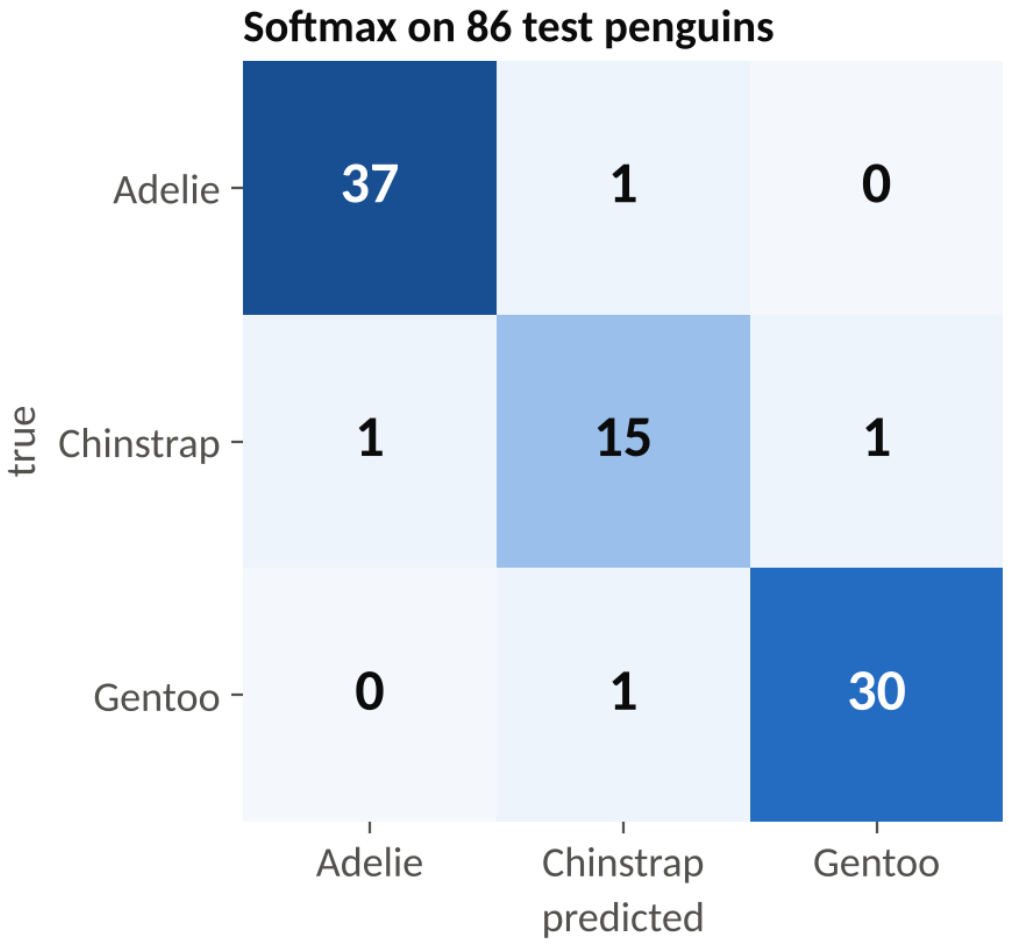


Watch for

- Uniform start: 1/3 each, loss log 3
- Three regions emerge
- Boundaries slide into the gaps
- The new penguin's bars separate

Gradient descent on the cross-entropy, η = 0.5, standardised features, 800 steps, no regularisation. Click to play (≈ 14 s).

Evaluating K classes: the confusion matrix



95.3%

overall test accuracy (82 / 86)

88%

Chinstrap recall (15 / 17): the class in the middle

44%

baseline: always predict Adelie (38 / 86)

Look per class and compare with the baseline — overall accuracy hides both.

In practice: a numerically stable softmax

```
z = np.array([1000., 1001.])
np.exp(z) / np.exp(z).sum()                  # [nan, nan] overflow

def softmax(Z):
    Z = Z - Z.max(axis=1, keepdims=True)     # max -> 0
    E = np.exp(Z)                            # all in (0, 1]
    return E / E.sum(axis=1, keepdims=True)

softmax(z[None, :])                          # [[0.269, 0.731]]
```

Never take log of a probability that may already be 0 — work with scores.



Why it works

Shift-invariance: subtracting the same number from every score leaves softmax unchanged.



In libraries

Log-softmax via log-sum-exp. PyTorch's CrossEntropyLoss expects raw scores (logits).

PART 05

Naive Bayes

The generative road: Bayes' theorem, a medical puzzle and a real spam filter



Bayes' theorem: updating beliefs with evidence

$$P(y \mid \mathbf{x}) = \frac{P(\mathbf{x} \mid y)\, P(y)}{P(\mathbf{x})}$$

$$\text{posterior} = \frac{\text{likelihood} \times \text{prior}}{\text{evidence}}$$

Prior P(y)

belief before seeing x — e.g. 13% of SMS are spam

Likelihood P(x | y)

how typical are these features for class y?

Evidence P(x)

same for every class — often ignored

Posterior P(y | x)

belief after seeing x — what we want

A puzzle: you test positive. Are you ill?



A disease affects **1%** of people screened.

The test detects **90%** of ill people.

It is also positive for **9%** of healthy people.

You test positive. What is the probability that you are ill?

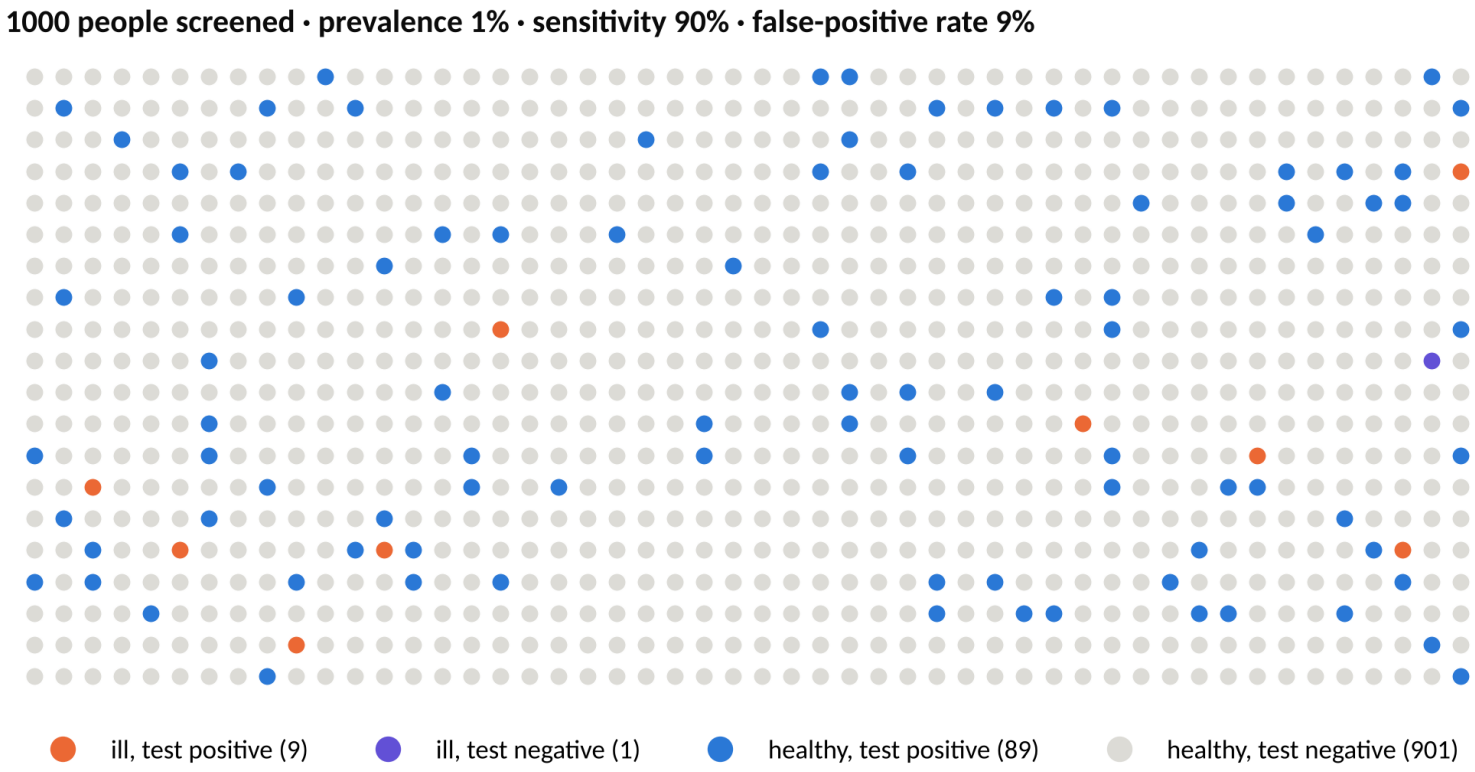
A $\approx 90\%$

B $\approx 81\%$

C $\approx 50\%$

D $\approx 9\%$

Answer: about 9% — count people, not percentages



9 / 98

of the people who test positive are ill

≈ 9.2%

= P(ill | positive)

$$P(\text{ill} \mid \text{positive}) = \frac{0.90 \times 0.01}{0.90 \times 0.01 + 0.09 \times 0.99} = \frac{0.009}{0.0981} \approx 0.092$$

Base-rate neglect

Rare class ⇒ most positives are false. Always use the prior.

Bayes as a classifier — and the problem with P(x | y)

The Bayes classifier

$\hat{y}$ = arg max over classes of P(x | y) · P(y). Optimal — if we knew the true distributions.

Easy part: the prior

Count: 747 of 5,572 SMS are spam ⇒ P(spam) ≈ 0.134.

Hard part: P(x | y)

2^{7387}

possible word patterns — a number with ~2,224 digits — to estimate from 4,179 training messages.

Even 30 yes/no features: $2^{30} \approx 1$ billion combinations. The curse of dimensionality.

The ‘naive’ assumption

$$P(\mathbf{x} \mid y) = P(x_1, \ldots, x_d \mid y) \approx \prod_{j=1}^{d} P(x_j \mid y)$$

$$\hat{y} = \arg\max_k \left[\log P(y = k) + \sum_{j=1}^{d} \log P(x_j \mid y = k) \right]$$

From 2^7387 to ~15,000 numbers

one small distribution per feature and class, each estimated by counting — training is one pass over the data

Why ‘naive’?

- Given the class, features are treated as independent
- Almost never true: ‘free’ and ‘prize’ co-occur
- Correlated evidence is double-counted

Why it still works

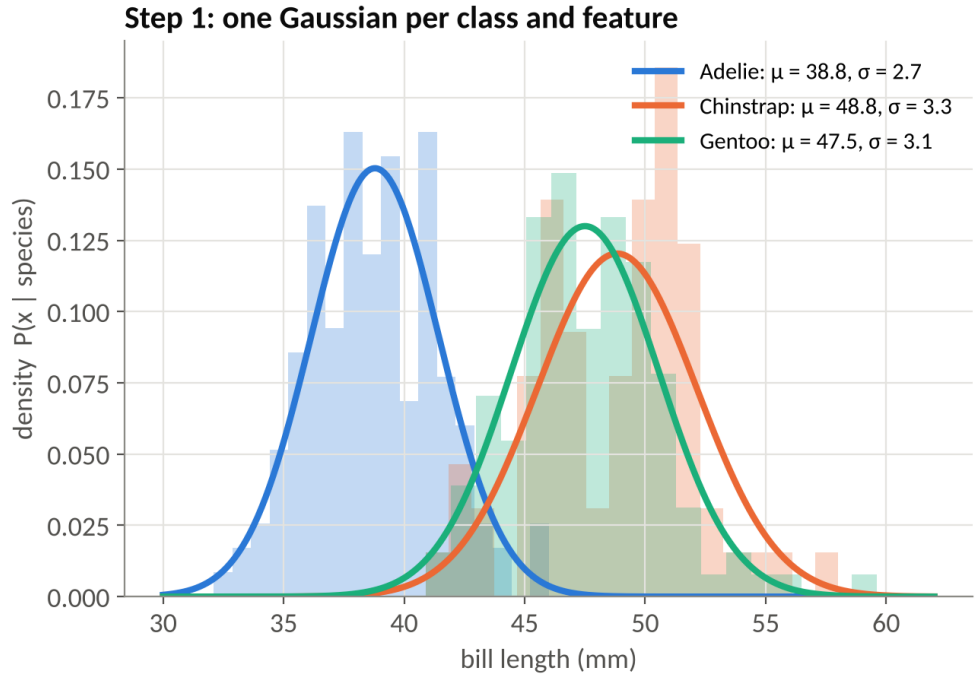
Classification needs the right winner, not the right probability.

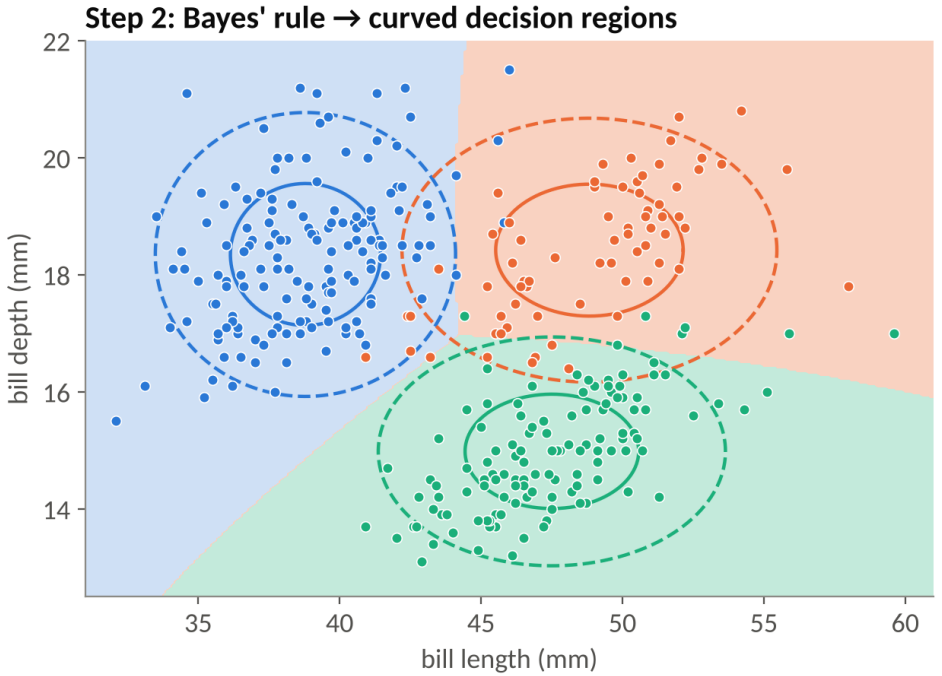
Gaussian Naive Bayes on the penguins

$$P(x_j \mid y = k) = \frac{1}{\sqrt{2\pi\sigma_{jk}^2}} \exp\left(-\frac{(x_j - \mu_{jk})^2}{2\sigma_{jk}^2}\right)$$

93.0%
Gaussian NB test accuracy

95.3%
softmax regression (Part 04)

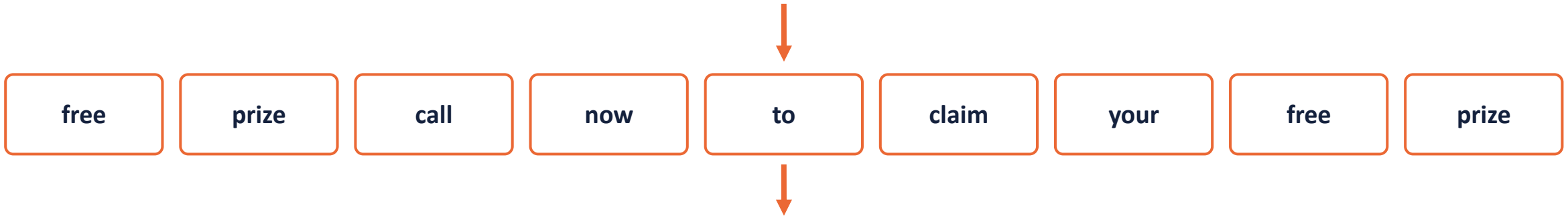




Text as numbers: the bag of words



“Free prize! Call now to claim your free prize”



word	free	prize	call	now	to	claim	your	lunch	…	(7,387)
count	2	2	1	1	1	1	1	0	…	0



Order is lost 'dog bites man' = 'man bites dog'



CountVectorizer lowercase · tokens ≥ 2 characters · no punctuation



Sparse 7,387 columns, ~10 non-zero per message

Multinomial Naive Bayes and Laplace smoothing

$$\hat{P}(w \mid c) = \frac{\text{count}(w, c) + \alpha}{\sum_{v \in V} \text{count}(v, c) + \alpha\,|V|}$$

The zero-count veto

"Did you claim it? Call me when you get home."

Unsmoothed: P(message | ham) = … × 0 × … = 0 ⇒ 100% spam.
With α = 1: P(spam) ≈ 0.0001.

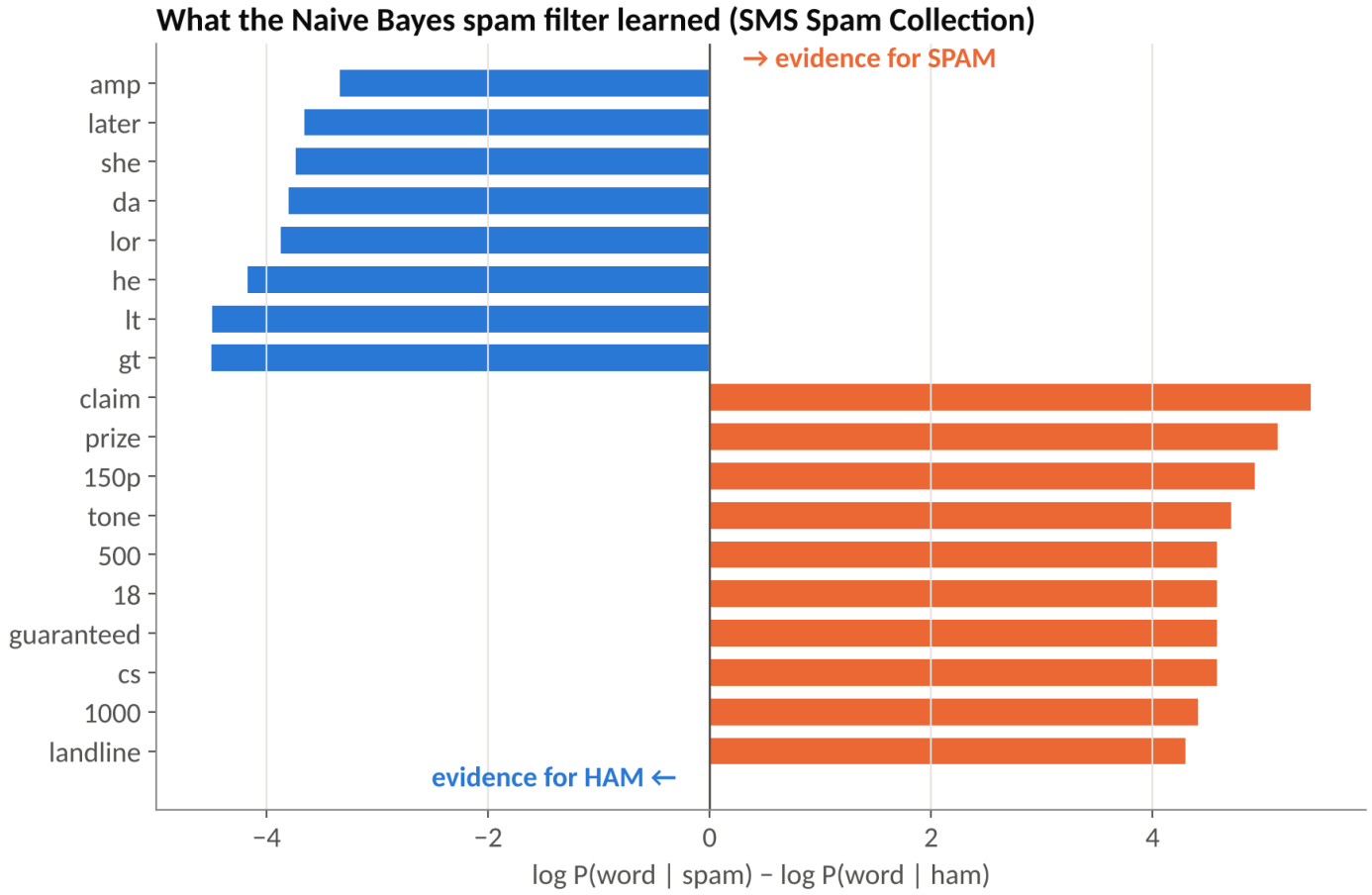
Training counts: 13,172 spam words, 46,991 ham words, |V| = 7,387.

Word	In spam	In ham	P(w | ham) raw	P(w | ham), α = 1
claim	85	0	0 ✗	1.8×10^{-5}
prize	63	0	0 ✗	1.8×10^{-5}
lunch	0	33	7.0×10^{-4}	6.3×10^{-4}
free	171	43	9.2×10^{-4}	8.1×10^{-4}

Fix: add-one (Laplace) smoothing

- Pretend every word was seen α = 1 more time per class
- Nothing is impossible; rare words barely change
- It's a prior (MAP) — MultinomialNB(alpha=1.0)

What the spam filter learned



Spam vocabulary

claim · prize · guaranteed · awarded · 150p · £1000 · landline

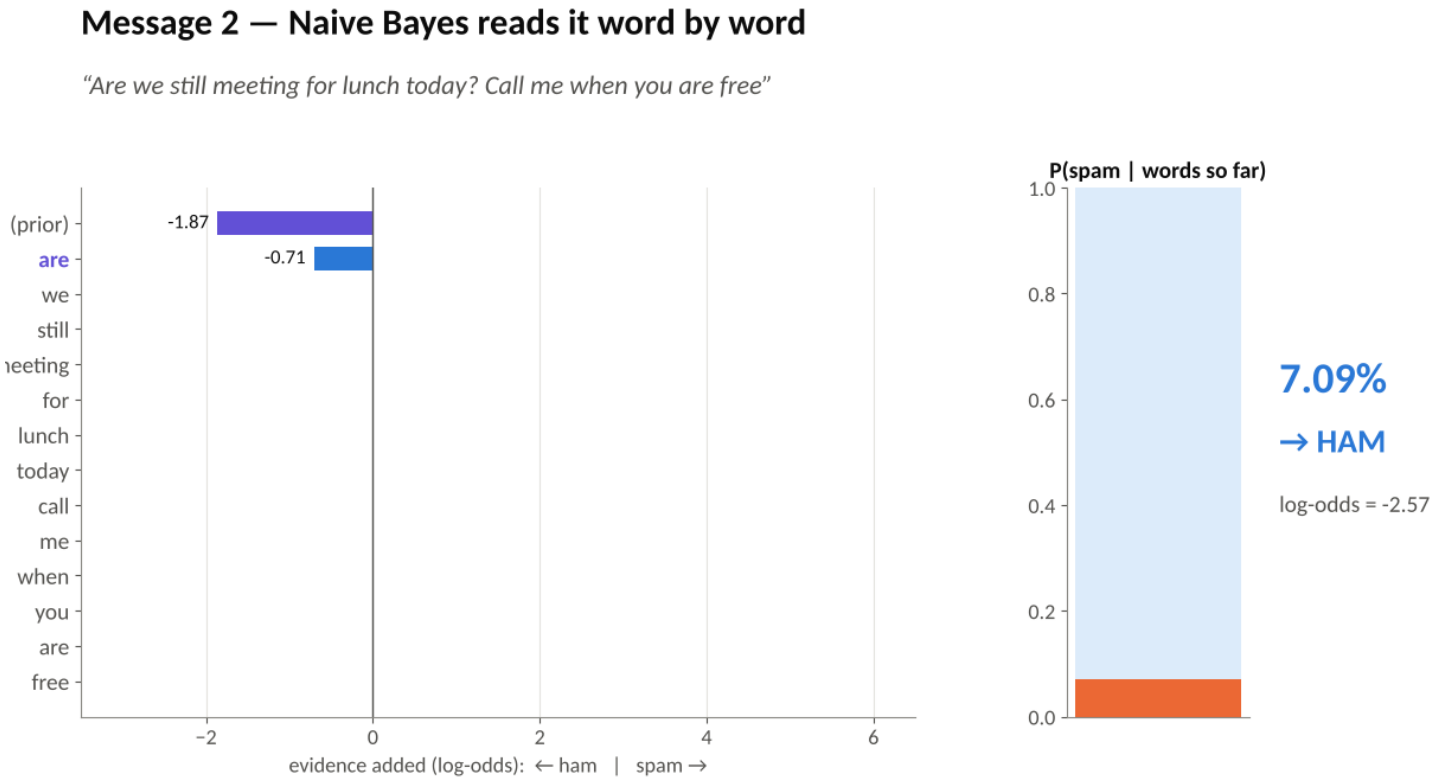
Ham vocabulary

he · she · later · lor · da — everyday chat and local slang

Data artefact!

lt · gt · amp = fragments of HTML codes like < — not words

Video: Naive Bayes reads a message word by word

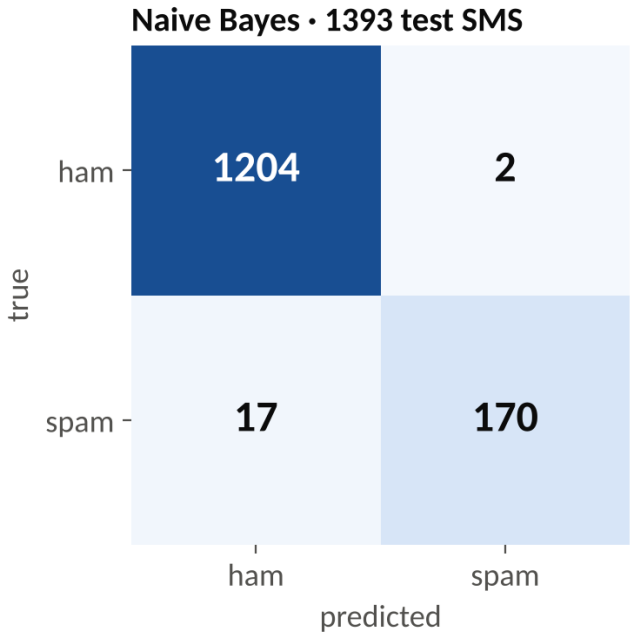


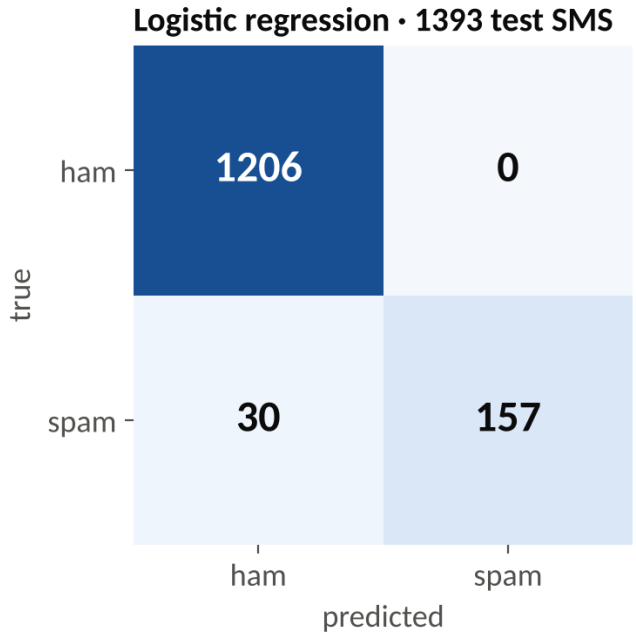
$$\log \frac{P(\text{spam} \mid \text{msg})}{P(\text{ham} \mid \text{msg})} = \log \frac{P(\text{spam})}{P(\text{ham})} + \sum_{w \in \text{msg}} \log \frac{P(w \mid \text{spam})}{P(w \mid \text{ham})}$$

Watch for

- Prior first: 13% spam
- 'winner' alone → 83%
- Message 2 has 'call' and 'free' — still ham
- Evidence adds up in log-odds (equation below the video)
- Multinomial NB, α = 1, 4,179 training SMS · click to play (≈ 14 s)

Results: Naive Bayes vs logistic regression on SMS





98.64%

Naive Bayes

97.85%

Logistic regression

86.58%

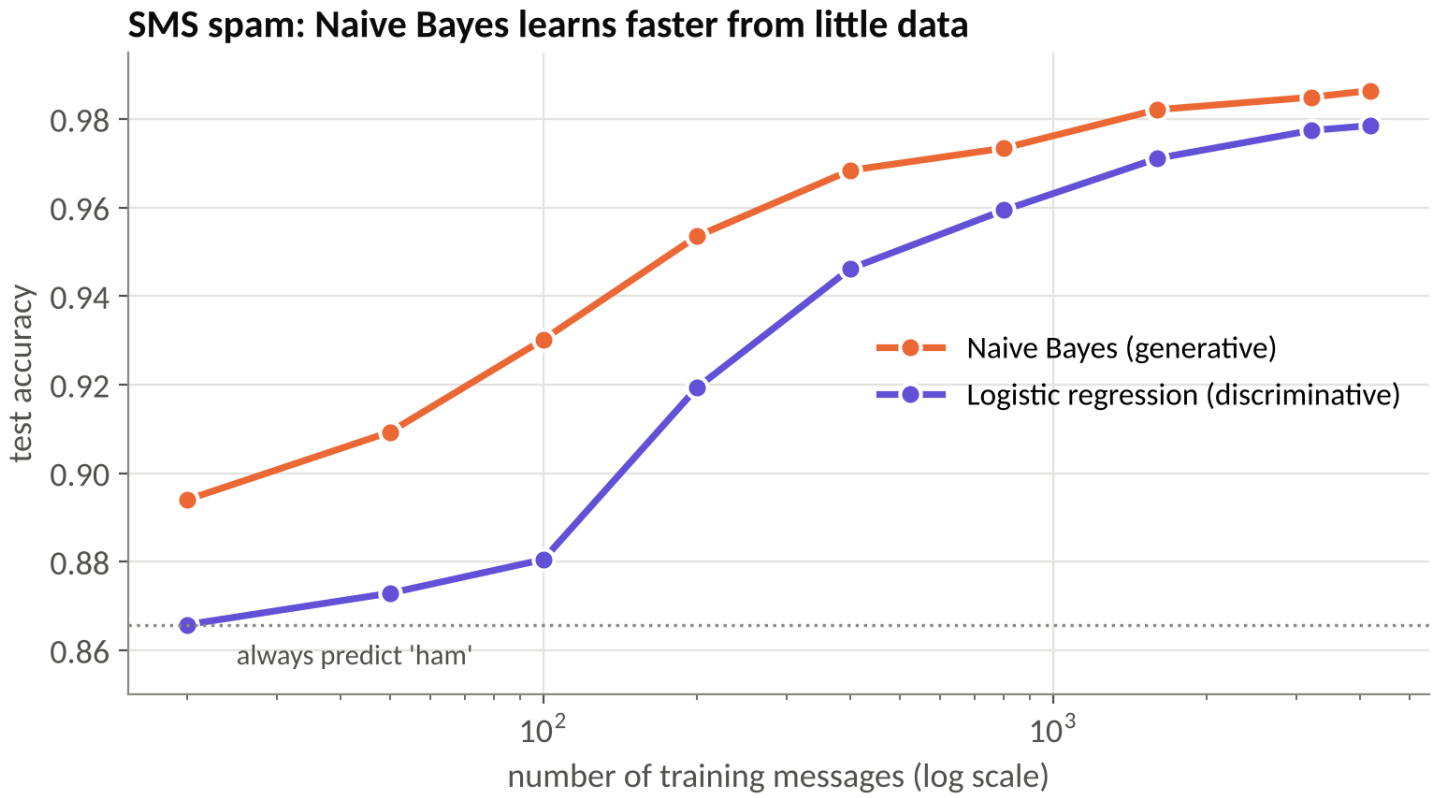
Always 'ham'

Which mistake matters?

A friend's message in the spam folder usually costs more than one spam in the inbox → tune the threshold for few false positives.

NB trains by counting — no learning rate, no iterations.

Generative vs discriminative: who needs less data?



Ng & Jordan (2002)

- Logistic regression: lower error with lots of data
- Naive Bayes: reaches its best with far fewer examples

93.0% vs 88.0%

NB vs LR with only 100 training messages

Strong assumptions win with little data — next week: the bias–variance trade-off.

Quick check: Naive Bayes by hand

	spam	ham
Prior P(y)	0.2	0.8
P('win' | y)	0.3	0.01
P('today' | y)	0.05	0.1

Message: ***"win today"*** — spam or ham, and with what probability?

```
spam:   0.2 × 0.3 × 0.05 = 0.003
```

```
ham:    0.8 × 0.01 × 0.1 = 0.0008
```

```
P(spam | msg) = 0.003 / 0.0038 ≈ 0.79
```

Odds view: 30 (win) × ½ (today) × ¼ (prior) = 3.75 → 3.75 / 4.75 ≈ 0.79

PART 06

In Practice

scikit-learn pipelines, real-world case studies, pitfalls and responsible use



Everything from today in scikit-learn

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

clf = make_pipeline(StandardScaler(),
                    LogisticRegression(C=1.0))
clf.fit(X_train, y_train)
p = clf.predict_proba(X_test)[:, 1]   # probabilities
y_hat = (p >= 0.3).astype(int)   # your threshold
np.exp(clf[-1].coef_)   # odds ratios
```

```
from sklearn.feature_extraction.text \
     import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

spam = make_pipeline(CountVectorizer(),
                     MultinomialNB(alpha=1.0))
spam.fit(msgs_train, y_train)
spam.predict(['WINNER! Claim your prize'])   # [1]

GaussianNB().fit(X_pen, species)   # no scaling needed
```



Pipelines prevent leakage: fit scalers and vocabularies on training data only



Multiclass is automatic: 3 species → softmax (multinomial)

Case studies: where these models run today



Spam filtering

- Bayesian filter learned from each user's mail
- Very few false positives
- Spread to mail software within a year or two

Paul Graham, “A Plan for Spam” (2002)



Ad-click prediction

- Logistic regression trained online
- Billions of sparse features, L1-style penalty
- Probabilities used to price ads

McMahan et al., Google, KDD 2013



Credit scoring

- Each feature adds ‘points’ = scaled weights
- Reasons for rejection must be explained
- Regulators can audit every coefficient

Logistic-regression scorecards

Logistic regression or Naive Bayes?

	Logistic regression	Naive Bayes
Models	P(y | x) directly (discriminative)	P(x | y) and P(y) (generative)
Assumes	log-odds linear in x	feature distributions + independence given y
Little data	needs more examples	**strong — learns fast**
Lots of data	**usually lower error**	assumptions limit it
Training	iterative optimisation	**one pass of counting**
Probabilities	**usually well calibrated**	often extreme — trust the ranking
Correlated features	shares weight between them	double-counts evidence
Interpretability	weights = log odds ratios	word log-ratios

Text + small data → start with Naive Bayes. Tabular data → regularised logistic regression. Always try both.

Six common pitfalls

Accuracy on imbalanced data

always-ham scores 87%; compare with the baseline

Blind 0.5 threshold

pick t from costs, on validation data

Leakage

fit scalers / vocabularies on training data only

Unscaled features (LR)

slow GD, unfair penalty — standardise

Trusting NB probabilities

0.9999 ≠ 99.99% — use the ranking

No regularisation

separable data → exploding weights

Responsible use: these probabilities affect people



Proxies

Dropping gender or ethnicity doesn't remove bias — postcode or names can stand in for them. Measure outcomes per group.



Explanations

People deserve reasons. Coefficients and word log-ratios give them almost for free.



Errors per group

High overall accuracy can hide low recall for a rare group — like Chinstrap.



Drift & humans in the loop

Spammers and customers change. Monitor, retrain, and let professionals decide high-stakes cases.

Summary: six ideas to take home

1 Probabilities first

Predict P(y | x); the threshold encodes the costs of mistakes.

2 Logistic regression

$P = \sigma(w^T x)$; log-odds linear; linear boundary; e^w = odds ratio.

3 Maximum likelihood

Make the data most probable. Gaussian → squared error; Bernoulli → log loss.

4 One gradient

X^T(prediction − truth). No closed form → GD. Separable → regularise.

5 Softmax

K scores → K probabilities; cross-entropy; stable with max-shift.

6 Naive Bayes

Bayes + independence; counting + smoothing; fast, small-data friendly.

A classifier is a probability model; its loss is the negative log-likelihood.

Exit quiz

1. What is σ(0)? Where is the decision boundary of logistic regression?
2. A standardised feature has weight w = 0.7. What happens to the odds when it rises by one standard deviation?
3. The true class receives probability 0.2. What is the log loss for this example?
4. The two classes are perfectly separable. What happens to the MLE weights, and how do we fix it?
5. Why does Naive Bayes need smoothing? What goes wrong with a word never seen in ham?

RESOURCES

Watch and read this week

Videos (≈ 45 min in total)

- StatQuest — Logistic Regression
- StatQuest — Maximum Likelihood
- StatQuest — Naive Bayes
- 3Blue1Brown — Bayes theorem (geometry of beliefs)

Reading

- ISLP — James et al., Ch. 4 (§4.1–4.4), free
- Jurafsky & Martin, Speech and Language Processing — NB & LR chapters, free
- Géron, Hands-On Machine Learning, Ch. 4: logistic & softmax regression

Minimum before next week: the maximum-likelihood video and ISLP §4.3.

Lab 3 preview: build it all yourself



Part A

Sigmoid & log loss

stable sigmoid, log loss, numerical gradient check



Part B

Logistic regression

gradient descent on penguins; compare with scikit-learn



Part C

Breast cancer

odds ratios, confusion matrix, choosing a threshold



Part D

Softmax regression

three species from scratch; one-hot; cross-entropy



Part E

Naive Bayes spam filter

counts, Laplace smoothing, log-space prediction

Every cell: what it does · your task · hints · expected result. CHECK cells print ✓when you are right.

Thank you — questions?

Next: Lab 3 — logistic regression, softmax and a Naive Bayes spam filter from scratch

Week 4: the bias–variance trade-off

Behnam Kiani · Kiani-bme@hotmail.com · office hours by appointment

Muddiest point

On a piece of paper or in the course Teams channel: what was the least clear part of today? One sentence. Not graded — it shapes next week.

